DERIVING DESCRIPTIONS OF POSSIBLE VALUES OF PROGRAM VARIABLES BY MEANS OF ABSTRACT INTERPRETATION

G. JANSSENS* AND M. BRUYNOOGHE†

▷ A PROLOG compiler specializes the code for unification between calls and clause heads as they appear in the program. This code could be further specialized, yielding more efficient code, if more precise information about possible values for actual arguments were available. This paper addresses the problem of gathering such information. It develops a method for obtaining descriptions of possible values of program variables. The method is based upon a framework for abstract interpretation. The descriptions can be regarded as extended modes or a kind of type information. An important issue in the method is the treatment of free variables and the sharing of free variables between different values of program variables. ◁

1. INTRODUCTION

A typical characteristic of a WAM-based PROLOG compiler is that it seldom uses the general unification algorithm for unifying a call and a head. Special instructions are generated which are adapted to the actual code appearing in calls and clause heads. Having information about the possible values for actual arguments of a call allows for further improvements of the code. It is well known that mode declarations allow for a substantial speedup. Having yet more detailed information allows for further improvements in performance as shown in [17].

This paper addresses the problem of deriving such detailed information. During execution, program variables become bound to terms which can be seen as their

* Supported by project RFO-AI-02: "Logic as a basis for artificial intelligence: control and efficiency of deductive interference -parallelism."

† Supported by the Belgian National Fund for Scientific Research.

Address correspondence to G. Janssens, Department of Computer Science, K. U. Leuven, Celestijnenlaan 200A, B-3001 Heverlee, Belgium.

Received May 1989; accepted January 1990.

©Elsevier Science Publishing Co., Inc., 1992
655 Avenue of the Americas, New York, NY 10010 0743-1066/92/$5.00

values. One way to describe sets of possible values of program variables is by a mode which distinguishes between different degrees of groundness, i.e. free, partially instantiated, and ground. We introduce type graphs to allow for more precise descriptions of possible values. Such type graphs not only give information about the degree of groundness but also about the names and positions of the functors occurring in the terms. During execution, a program variable has different values at different points. Initially it is free, and it becomes gradually more and more instantiated. At every point, we can consider the set of possible values, and we can approximate this set by a type graph. We give a method to compute type graphs associated with program variables at different points which are safe approximations of the sets of possible values.

The method is an application of the framework for abstract interpretation of logic programs described in [3]. Consequently, to show that our method indeed computes safe approximations, it suffices to show that the type graphs making up the abstract domain have the required algebraic structure and that the primitive abstract operations required by the abstract interpretation procedure satisfy the conditions stated in [3].

The type graphs by themselves are not always enough to obtain sufficient precision. Consider for example a case where we have three program variables, X, Y, and Z, and a point in the execution where the values of Z and X share a free variable. Unifying X and Y changes the values of X and Y but also the value of Z. The type graph for Z remains correct in the case it is *substitution-closed* (i.e., when a term t has a type T, then also all instances of t have type T). However, substitution-closed types are not very precise; for instance, one cannot distinguish a free variable from the universe of all terms, as all terms are instances of a free variable. To handle types which are not substitution-closed, it becomes necessary to have a sufficiently precise treatment of sharing information, i.e., to know whether the type of Z needs to be updated after unifying X and Y. In [5] we have briefly discussed substitution-closed types; in [4] we have refined our representation and included sharing information. In [10] and [9] the sharing representation has been improved by taking into consideration whether a term contains several occurrences of the same free variable, and a complete formalization, including proofs, of both type systems is given. The current paper summarizes this work and attempts to stress the underlying intuitions.

The automatic derivation of modes was first studied by Mellish [18]. The problems caused by the sharing of free variables between terms were properly addressed for the first time by Debray [6]. Mode inferencing is almost the simplest application of abstract interpretation one can consider and has been used by several authors to illustrate their approach (a.o. [3, 15]). It has reached the state where it is applied in experimental compilers [29].

Our type graphs are quite different from most of the works on types in logic programming, where types are usually seen as descriptions of sets of ground terms and are used either in a prescriptive way (yielding a typed language) or in a descriptive way (characterizing the success typing of a predicate). See a.o. [7, 12, 14, 19, 22, 33, 34, 35] for works on types. Somogyi [25] has used modes as tags to types, and also Bansal and Sterling [2] have combined modes and types in a schema for abstract interpretation. The handling of free variables shared between terms has been considered not only in the case of deriving modes, but also in the context of occur check reduction [26] and the exploitation of AND parallelism [23, 32, 8, 21].

Section 2 starts with a sketch of the underlying framework for abstract interpretation. Also the requirements on the abstract domain and the primitive operations needed to complete an application are described. We motivate our application in Section 3 with a discussion of the code generation process and of the usefulness of detailed information about possible values of program variables. The substitution-closed rigid types are developed in Section 4. The subclasses of normal and restricted types are defined, and several operations manipulating types are described. The abstract substitutions making up the abstract domain are introduced in Section 5. Besides a type component, the abstract substitutions also have a component indicating whether subterms in the values of different program variables always have the same value. Section 5 finishes with the development of the algebraic structure on the abstract domain as required by the framework. The application based on rigid types is completed in Section 6 with a description of the primitive operations. Section 7 uses an example to evaluate the rigid types and points out the need for a richer system, the integrated types. Section 8 introduces integrated types which are not substitution-closed, extends abstract substitutions with sharing information, and discusses the changes needed to handle these richer types. Section 9 sums up our achievements, discusses the practical usability of the work, and sketches areas requiring further investigations.

All technical details (algorithms, proofs, etc.) omitted in this paper can be found in [11], which is an excerpt of the relevant chapters of [9].

2. THE FRAMEWORK FOR ABSTRACT INTERPRETATION

2.1. Introduction

The framework for abstract interpretation we use is developed in [3]. Here, we review the abstract interpretation procedure for the construction of an AND-OR graph and attempt to give the intuition behind it. This graph is used to organize the information gathered by the abstract interpretation process.

To optimize the compilation of procedure calls, we have to gather the information concerning the possible values of the variables in the call. Considering a particular state in an SLD derivation, i.e. $\leftarrow (Q_1, Q_2, \ldots, Q_n)\sigma$, the information of interest is $\sigma|_{\mathrm{var}(Q_1)}$,[1] the restriction of the accumulated substitution σ to the variables of the call to be executed in the next step. So let us review some steps in the SLD derivation of a query $\leftarrow A_1, \ldots, A_q$ (under a left to right computation rule).

Assume we have reached the goal statement

$$\leftarrow (A_i, A_{i+1}, \ldots, A_q)\sigma_i \tag{1}$$

with σ_i the substitution accumulated in the derivation. The domain of this substitution consists of the variables occurring either in the initial query or in the renamed clauses used during the derivation. At this point, a renamed clause $B \leftarrow B_1, \ldots, B_p$ is selected and $\theta_k = \mathrm{mgu}(A_i\sigma_i, B)$ is computed, and the next goal statement is

$$\leftarrow (B_1, \ldots, B_p, A_{i+1}, \ldots, A_q)\sigma_i\theta_k. \tag{2}$$

[1] For any syntactic object o, $\mathrm{var}(o)$ denotes the set of variables occurring in o.

Applying the same procedure for a number of additional derivation steps eventually leads to a goal statement

$$\leftarrow (A_{i+1},\ldots,A_q)\sigma_{i+1}, \tag{3}$$

where $\sigma_{i+1} = \sigma_i\theta_k\theta_{k+1}\cdots\theta_l$ with $\theta_{k+1}\cdots\theta_l$ the mgu's resulting from these additional steps. Observe that there is a refutation of

$$\leftarrow (B_1,\ldots,B_p)\theta_k \qquad \text{with a sequence of mgu's } \theta_{k+1}\cdots\theta_l. \tag{4}$$

This derivation schema is used as the basis of PROLOG implementations and is also the basis of the abstract interpretation framework of [13]. However, we prefer a different schema as basis which does not use accumulating substitutions but substitutions restricted to the variables of a single clause. Some of the (subjective) advantages are:

It is easier to compare different activations of the same clause because the domains of the substitutions are each other's renamings. Such comparisons are crucial.

It becomes possible to organize the gathered information in an AND-OR graph. These graphs can be considered as representing a set of proof trees [3] and give rise to multiple specializations [30] in a very natural way.

The distinguishing feature of the new schema is that only goal statements of the form $\leftarrow (A_i,\ldots,A_p)\tau$ are considered, where:

The sequence of literals $(A_i,\ldots,A_p)$ either is the tail of a body of a clause $A \leftarrow A_1,\ldots,A_p$ or is the tail of the initial query $\leftarrow A_1,\ldots,A_p$.

The domain of the substitution is a subset of the variables occurring in respectively the clause or the query.

The new schema is equivalent to the old one in the following sense:

For a query $\leftarrow A_1,\ldots,A_p$ with a refutation with accumulated substitution σ and answer substitution $\sigma_r = \sigma|_{\mathrm{var}(A_1,\ldots,A_p)}$, the new schema also yields the answer substitution σ_r.

The new schema also considers subderivations starting with goal statements of the form $\leftarrow (B_1,\ldots,B_p)\theta$ when $B \leftarrow B_1,\ldots,B_p$ is a clause of the program and θ a substitution over the variables of that clause. Such a subderivation returns an answer τ iff there is a refutation of $\leftarrow (B_1,\ldots,B_p)\theta$ with accumulating substitution σ and $\tau = \theta\sigma|_{\mathrm{var}(B \leftarrow B_1,\ldots,B_p)}$.

To sketch the new schema, we consider again a query $\leftarrow A_1,\ldots,A_q$ and assume we have reached a state

$$\leftarrow (A_i, A_{i+1},\ldots,A_q)\tau_i \tag{1'}$$

with τ_i a substitution over variables of $A_1,\ldots,A_q$. The derivation chain is suspended and a new subderivation is initiated as follows: Let $B \leftarrow B_1,\ldots,B_p$ be a renamed clause, and θ_k the mgu of $A_i\tau_i$ and B. The initial goal statement of the subderivation is

$$\leftarrow (B_1,\ldots,B_p)\theta_k|_{\mathrm{var}(B \leftarrow B_1,\ldots,B_p)}. \tag{2'}$$

In anticipation of one of the steps in the abstract interpretation procedure and to conform with the procedural interpretation of logic programs, we call this step *procedure entry*.

Continuing the subderivation starting with (2'), we eventually obtain the empty goal statement and an answer substitution θ with a subset of $\mathrm{var}(B \leftarrow B_1, \ldots, B_p)$ as domain. At this point we apply an operation called *procedure exit*. We compute τ, the mgu of $A_i\tau_i$ and $B\theta$ and derive

$$\leftarrow (A_{i+1}, \ldots, A_q)\tau_{i+1} \tag{3'}$$

with $\tau_{i+1} = \tau_i\tau|_{\mathrm{var}(A_1, \ldots, A_q)}$ (in general we restrict the substitutions to the variables of the clause or query that $A_{i+1}, \ldots, A_q$ is part of).

To obtain some evidence for the claim that the new derivation schema delivers the same answers as the original schema, we compare the two fragments of derivations.

Assume τ_i in (1') is the restriction of σ_i in (1) to the variables of $A_1, \ldots, A_q$. So $(A_1, \ldots, A_q)\sigma_i = (A_1, \ldots, A_q)\tau_i$, and in particular $A_i\sigma_i = A_i\tau_i$. Using the same clause $(B \leftarrow B_1, \ldots, B_p)$ in both derivations, we have the same mgu θ_k. Assuming correctness of the scheme for the subderivation starting with $\leftarrow (B_1, \ldots, B_p)\theta_k|_{\mathrm{var}(B \leftarrow B_1, \ldots, B_p)}$ and given that there is a refutation of $\leftarrow (B_1, \ldots, B_p)\theta_k$ with accumulated substitution $\theta_{k+1} \cdots \theta_l$ (4), the subderivation returns an answer $\theta = \theta_k\theta_{k+1} \cdots \theta_l|_{\mathrm{var}(B \leftarrow B_1, \ldots, B_p)}$. So $B\theta = B\theta_k\theta_{k+1} \cdots \theta_l$.

In order to show that (3) = (3'), we have to prove that $\tau_{i+1} = \sigma_{i+1}|_{\mathrm{var}(A_1, \ldots, A_q)}$. For the left hand side we consecutively apply the definitions of τ_{i+1}, τ_i, τ, and θ to derive the equalities

$$\begin{aligned}
\tau_{i+1} &= \tau_i\tau \\
&= \sigma_i|_{\mathrm{var}(A_1, \ldots, A_q)}\, \mathrm{mgu}(A_i\sigma_i, B\theta) \\
&= \sigma_i|_{\mathrm{var}(A_1, \ldots, A_q)}\, \mathrm{mgu}(A_i\sigma_i, B\theta_k\theta_{k+1}, \ldots, \theta_l).
\end{aligned}$$

Since $\theta_k = \mathrm{mgu}(A_i\sigma_i, B)$, we have that $B\theta_k = A_i\sigma_i\theta_k$, so

$$\tau_{i+1} = \sigma_i|_{\mathrm{var}(A_1, \ldots, A_q)}\, \mathrm{mgu}(A_i\sigma_i, A_i\sigma_i\theta_k\theta_{k+1}, \ldots, \theta_1).$$

Now, for any term t and any substitution ψ, $\mathrm{mgu}(t, t\psi) = \psi|_{\mathrm{var}(t)}$, so that we get

$$\begin{aligned}
\tau_{i+1} &= \sigma_i|_{\mathrm{var}(A_1, \ldots, A_q)}\theta_k\theta_{k+1}, \ldots, \theta_1|_{\mathrm{var}(A_i\sigma_i)} \\
&= \sigma_i|_{\mathrm{var}(A_1, \ldots, A_q)}\theta|_{\mathrm{var}(A_i\sigma_i)} \\
&= \sigma_i\theta|_{\mathrm{var}(A_1, \ldots, A_q)} \\
&= \sigma_{i+1}|_{\mathrm{var}(A_1, \ldots, A_q)},
\end{aligned}$$

because θ only affects the variables in $A_i\sigma_i$, and by definition of σ_{i+1}.

The main differences with the original schema are:

Call and clause heading are unified twice: once upon entry, once upon exit.

Only goal statements of the form $(B_i, \ldots, B_p)\theta$ with $B_i, \ldots, B_p$ a tail of a clause body (query) and θ a substitution over the variables of that clause (query) are considered.

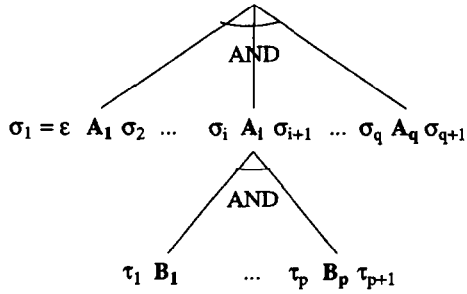


FIGURE 1. Part of tree representation of a derivation for a query $\leftarrow A_1, \ldots, A_q$.

The derivation can be graphically represented as in Figure 1. Substitutions are over the variables of the clause or query they adorn. The substitution to the left of a call contains the values of the variables of the clause (query) at the point the call is selected for execution. The substitution to the right of the last call (e.g. σ_{q+1}, τ_{p+1}) is the answer; it shows the value of the variables of the clause (or query) after completing the computation for the clause body (query).

Our interest is not in one derivation originating from a single query, but in all derivations originating from a set of queries, i.e. all queries $\leftarrow (A_1, \ldots, A_q)\theta$ with θ an element of the set of substitutions Θ. As the sets of interest are often infinite, we need in fact finite descriptions of safe approximations of sets of substitutions. We call them abstract substitutions and use the symbol β to represent them.

A concretization function γ gives a meaning to an abstract substitution by mapping it into a set of substitutions: $\gamma(\beta) = \Theta$. This mapping can be extended to expressions (clauses, terms, etc.) as follows: $\gamma(E\beta)$ denotes the set of expressions $E\theta$ such that $\theta \in \gamma(\beta)$.

The tree representation of a single derivation as in Figure 1 is generalized into an AND-OR graph as in Figure 2. The graph is a finite representation of an infinite tree in the same way as an (infinite) rational tree can be represented as a finite graph. OR nodes are introduced because a call can unify with several clause headings. Substitutions are replaced by abstract substitutions. Roughly speaking, for every goal statement $\leftarrow (B_i, \ldots, B_p)\sigma_i$ which can appear as goal statement in a

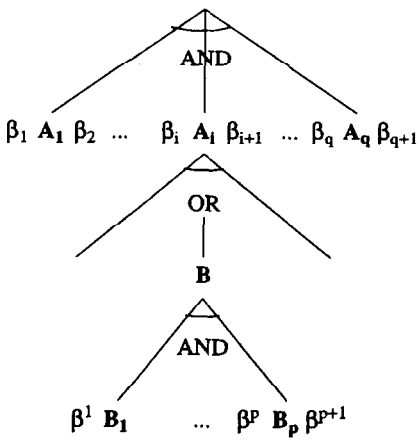


FIGURE 2. Part of AND-OR graph for the query $\leftarrow A_1, \ldots, A_q$.

subderivation, one expects an instance of the clause (query) having $B_i, \ldots, B_p$ as tail with B_i adorned to the left with β_i such that $(B_i, \ldots, B_p)\sigma_i \in \gamma((B_i, \ldots, B_p)\beta_i)$. So, in a complete and correct AND-OR graph, the set $\gamma(B_i\, \beta_i)$ with β_i adorning B_i to the left is a safe approximation of the set of calls which can occur at run time, and the information in β_i can be used to optimize the compilation of the call B_i (in the case that the graph contains several instances of B_i, one either takes the union of the different sets, or introduces different versions of the predicate—multiple specialization [30]—using the structure of the graph as a guide). β_i is called the *abstract call substitution* of the call B_i, as it describes the set of instances for which the predicate is called. It is also the *abstract success substitution* of B_{i-1}, as it also describes the possible values for the clause variables immediately after successful completion of the calls to B_{i-1}.

Notice that the abstract substitutions describe sets of substitutions over the variables of the clause (query) they annotate. As a consequence, it is easy to compare abstract substitutions adorning different instances of the same clause. This facilitates the treatment of recursion, as can be seen in the procedure below.

2.2. *Overview of the Framework*

Each application requires the design of the application (domain)-dependent components, namely the abstract domain and the abstract interpretation operations. They are used in the abstract interpretation procedure, which is domain-independent.

2.2.1. Abstract Domain. The set of variables in the clause/query that an abstract substitution β adorns is called the *domain D* of that abstract substitution. A_D is the set of all abstract substitutions with the same domain. We use the convention that elements of A_D are denoted by β or δ, with or without subscripts or superscripts. The abstract interpretation framework requires A_D to have some algebraic structure. In fact, one can slightly relax the conditions imposed by [3]. It suffices to have:

A reflexive transitive relation $\leq$ (a preorder) with the property that

$\forall \beta, \delta \in A_D$: if $\beta \leq \delta$ then $\gamma(\beta) \subseteq \gamma(\delta)$.

So there is also an equivalence relation $\equiv$ with the property that

$\forall \beta, \delta \in A_D$: if $\beta \equiv \delta$ then $\gamma(\beta) = \gamma(\delta)$.

($\leq$ modulo equivalence classes is a partial order).

An upper bound operator[2] upp with the property that

$\forall \beta_1, \beta_2 \in A_D$: $\exists \text{upp}(\beta_1, \beta_2) \in A_D$:

$\beta_1 \leq \text{upp}(\beta_1, \beta_2)$ and $\beta_2 \leq \text{upp}(\beta_1, \beta_2)$.

A maximal element $\beta_{\max}$ such that $\forall \beta \in A_D$: $\beta \leq \beta_{\max}$.

A minimal element $\perp$ such that $\gamma(\perp) = \varnothing$ and $\forall \beta \in A_D$: $\perp \leq \beta$.

[2] Commutativity and associativity of the upper bound are desirable. To obtain as much precision as possible, upp should return a value as small as possible, e.g. the least upper bound in case the abstract domain is a lattice.

A subset F_D of A_D with the property that F_D has no ascending chain for $\leq$ (an ascending chain is an infinite sequence of elements $\{a_1, a_2, \ldots, a_n, \ldots\}$ such that $a_1 < a_2 < \cdots < a_n < \cdots$) and F_D includes $\beta_{\max}$ and $\perp$. The existence of F_D guarantees termination of the abstract interpretation procedure. There also must be an operator $R: A_D \to F_D$ with the property that $\beta \leq R(\beta)$. Note that the constant function f such that $f(\beta) = \beta_{\max}$ is a possible definition for R, but the loss of precision would be unacceptable.

2.2.2. Abstract Interpretation Operations. Compared with [3], some simplifications are possible because the atoms are assumed to be of the form $P(Y_1, \ldots, Y_n)$ with the Y_i distinct variables (a similar assumption is made for calls to the explicit unification $=/2$). Clauses can be transformed into such a form through the introduction of explicit calls to $=$, which is considered as a built-in. Throughout this paper, we assume clauses to be in this form.

First, we introduce some auxiliary components.

(1) Procedure-entry(P, β_{in}). Applied to a call P with abstract call substitution β_{in}, Procedure-entry extends the abstract AND-OR graph, as shown in Figure 3(a), and computes $\beta_1^{\text{in}}, \ldots, \beta_m^{\text{in}}$. The domain of β_j^{in} is the set of variables in the jth clause. It is convenient to consider also β_{in}^r with the domain restricted to the variables of the call P. If P is a recursive predicate, we require that β_{in}^r belongs to F_D.[3] Sufficient conditions for correctness of Procedure-entry are:

if $\tau \in \gamma(\beta_{\text{in}})$ then $P\tau \in \gamma(P\beta_{\text{in}}^r)$;
if $\tau \in \gamma(\beta_{\text{in}}^r)$ and θ is a mgu of $P\tau$ and H_j (with H_j the head of a properly renamed clause), then

$$\left(H_j \leftarrow B_1^j, \ldots, B_{j_n}^j\right)\theta \in \gamma\left(\left(H_j \leftarrow B_1^j, \ldots, B_{j_n}^j\right)\beta_j^{\text{in}}\right).$$

(2) Procedure-exit($P, \beta_{\text{in}}, \{\ldots, \beta_j^{\text{out}}, \ldots\}$). Procedure-exit is shown in Figure 3(b). It can be applied when the final abstract success substitution β_j^{out} is obtained for each clause defining P. In a first step, β_{out}^r, with the variables of P as domain, is computed.[4] Next β_{out} is computed. This last step is called the *extension*. Sufficient conditions for correctness are:

if

$$\exists \tau_j \in \gamma\left(\beta_j^{\text{out}}\right) \quad \exists \tau_{\text{in}} \in \gamma(\beta_{\text{in}}) \quad \exists \theta \text{ over var}(P\tau_{\text{in}}): \qquad P\tau_{\text{in}}\theta = H_j\tau_j$$

(with H_j the head of a properly renamed clause), then $\exists \tau \in \gamma(\beta_{\text{out}}^r)$: $P\tau = P\tau_{\text{in}}\theta$;
if

$$\exists \tau_{\text{in}} \in \gamma(\beta_{\text{in}}) \quad \exists \theta \text{ over var}(P\tau_{\text{in}}): \qquad P\tau_{\text{in}}\theta \in \gamma(P\beta_{\text{out}}^r),$$

then $(H \leftarrow B_1, \ldots, P, \ldots, B_n)\tau_{\text{in}}\theta \in \gamma((H \leftarrow B_1, \ldots, P, \ldots, B_n)\beta_{\text{out}})$.

[3] This is not essential but improves performance; the β_{in}^r defined in the application are members of A_D; we assume that Procedure-entry uses $R(\beta_{\text{in}}^r) \in F_D$ when handling a recursive predicate.

[4] Note that in the case of a recursive predicate P, β_{out}^r is not required to belong to F_D, although β_{in}^r is.

(3) Abstract-interpretation-built-in(P, β_{in}). This is shown in Figure 3(c). It consists of three steps. First β^r_{in} is computed with its domain restricted to the variables in the built-in and belonging to A_D. Then β^r_{out} is computed, and extension, defined in Procedure-exit, yields β_{out}. Correctness of the first two steps requires:

if $\tau \in \gamma(\beta_{in})$ then $P\tau \in \gamma(P\beta^r_{in})$;
if $\tau_{in} \in \gamma(\beta^r_{in})$ and executing $P\tau_{in}$ instantiates the call by θ, then $P\tau_{in}\theta \in \gamma(P\beta^r_{out})$.

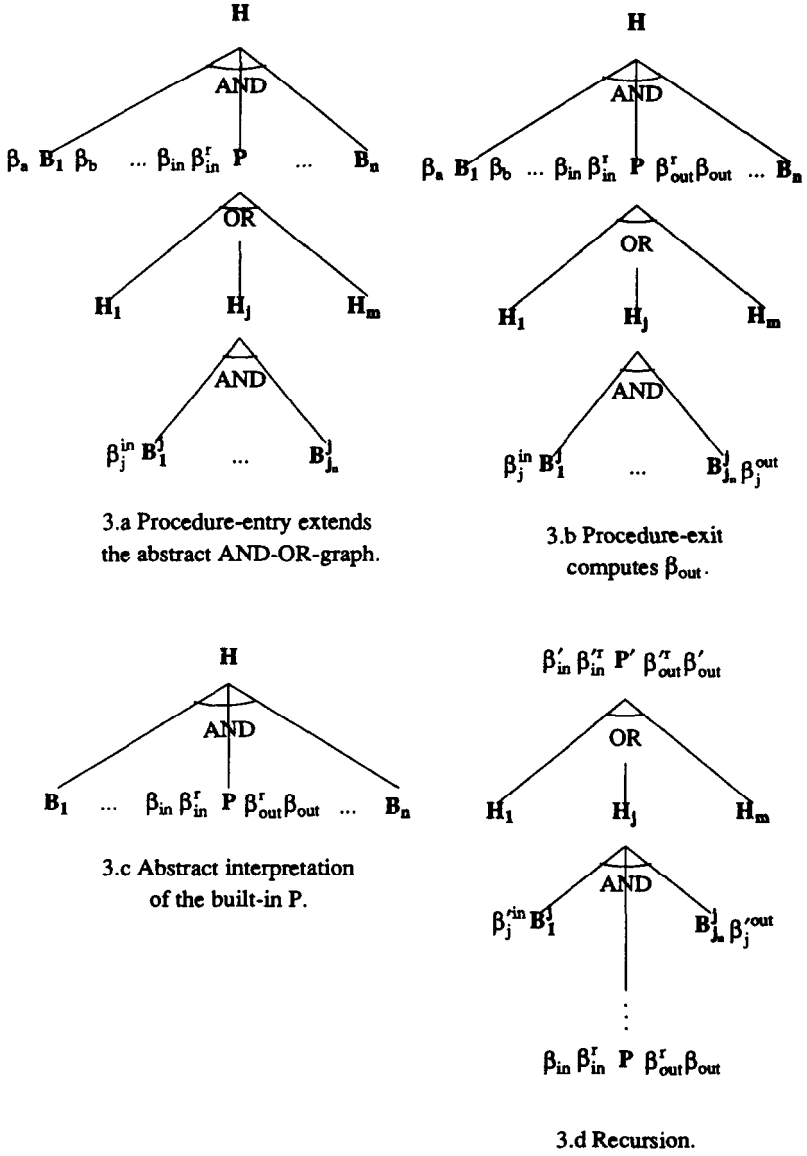


FIGURE 3. The primitive steps of the abstract interpretation procedure.

2.2.3. Abstract Interpretation Procedure. The kernel of the framework is the procedure Abstract-interpretation-call(P, β_{in}), which computes β_{out}, the abstract success substitution of P with the same domain as β_{in}. We distinguish the cases:

Case 1. If P is a built-in, then apply Abstract-interpretation-built-in(P, β_{in}).

Case 2. If P has no ancestor node with a call for the same predicate, then apply

Procedure-entry(P, β_{in}).
Apply Abstract-interpretation-call on the calls of the created bodies, always working on a call whose abstract call substitution is known.
Apply Procedure-exit($P, \beta_{in}, \{\ldots, \beta_j^{out}, \ldots\}$).

Case 3. If P has an ancestor node with a call P' to the same predicate [see Figure 3(d)] and if they have the same abstract call substitution $\beta_{in}^r \equiv \beta'^r_{in}$ (up to renaming of variables), then β_{out}^r is a renaming of β'^r_{out} which is computed by the first step of Procedure-exit($P', \beta'^r_{in}, \{\ldots, \beta'^{out}_j, \ldots\}$) using $\perp$ for the branches j for which β'^{out}_j cannot be computed. Next, extension can compute β_{out}, and the processing of call P is finished. At some point a Procedure-exit will again be applied on P' and a new value will be obtained for β'^r_{out}. Let β_1 be the old and β_2 be the new value. The computation proceeds as usual in case $\beta_2 \leq \beta_1$ and node P refers back to node P'. Otherwise, the computation starts over at the call P using $R(\text{upp}(\beta_1, \beta_2))$ as the new value for β_{out}^r and β'^r_{out}.

Case 4. If P has an ancestor node with a call P' to the same predicate but it is not the case that $\beta_{in}^r \equiv \beta'^r_{in}$,[5] then if $\beta_{in}^r \leq \beta'^r_{in}$, proceed as in case 3. Otherwise, recompute the subtree of P' using a new value for β'^r_{in} the value $R(\text{upp}(\beta_{in}^r, \beta'^r_{in}))$.

Algorithm Abstract-interpretation-procedure(Q, β).

Initialization: The abstract AND-OR graph for a set of queries $Q\theta$ is initialized with a root representing the call Q and adorned to the left with an abstract call substitution β such that every query $Q\theta \in \gamma(Q\beta)$.

Abstract-interpretation-call(Q, β).

For proofs of correctness and termination of the above procedure, as well as for further details, the reader is referred to [3].

The different correctness conditions express the requirements for the abstract operations to be safe approximations of the corresponding concrete operations in the new derivation schema. In the case of Procedure-entry, one considers all goal statements $\leftarrow (P, \ldots)\tau$ with $\tau \in \gamma(\beta_{in})$. Using the renamed clause $H_j \leftarrow B_1^j, \ldots$, the initiated subderivation is $\leftarrow (B_1^j, \ldots)\theta$ with θ the mgu of $P\tau$ and H_j restricted to the variables of the clause. The condition expresses that this θ must be a

[5] One might also apply case 2, although not without limit, as the graph must remain finite; see [3]. However, experience shows that this substantially increases computation time while resulting in uninteresting cases of multiple specialization [9].

member of $\gamma(\beta_j^{in})$. The case of Procedure-exit is similar but more complex. Again one considers all goal statements of the form $\leftarrow (P, Q, \ldots)\tau_{in}$ with $\tau_{in} \in \gamma(\beta_{in})$. For such a goal statement, a subderivation $\leftarrow (B_1^j, \ldots)\sigma$ has been initiated. Now, not all elements of $\gamma(\beta_j^{out})$ can be answers for that subderivation, but only the substitutions τ_j such that $H_j\tau_j$ is an instance of $P\tau_{in}$, i.e. $H_j\tau_j = P\tau_{in}\theta$. If such a τ_j is indeed an answer, then concrete procedure exit computes $\leftarrow (Q, \ldots)\tau_{out}$. The condition expresses that this τ_{out} must be a member of $\gamma(\beta_{out})$. The case of abstract interpretation of a built-in is obvious.

3. TARGET OF THE ANALYSIS

In order to judge the appropriateness of the derived information, it is necessary to take into account its intended use: we want to gather information about the execution which makes it possible to specialize the generated code, thereby increasing its performance. In order to have an idea of the kind of information which is useful for code generation, we point out the principal optimization issues. We assume some acquaintance with the execution mechanism of PROLOG and with WAM-based compilers. The following examples will be used throughout the text.

append(*nil*, *L*, *L*).

append(*X*.*U*, *V*, *X*.*W*) :– *append*(*U*, *V*, *W*).

insert(*E*, *nil*, *t*(*nil*, *E*, *nil*)).

insert(*E*, *t*(*L*, *F*, *R*), *t*(*NL*, *F*, *R*)) :– $E \leq F$, *insert*(*E*, *L*, *NL*).

insert(*E*, *t*(*L*, *F*, *R*), *t*(*L*, *F*, *NR*)) :– $E > F$, *insert*(*E*, *R*, *NR*).

3.1. Some Characteristics of WAM-based PROLOG Systems

PROLOG execution is based on unification and backtracking. In this context, it is important to keep in mind that:

- A *choice point* has to be created for each call which has the potential of unifying with the head of more than one clause. It contains the information needed for backtracking, and its creation is time-consuming.
- As PROLOG is an untyped language, the type of a value must explicitly be represented; the value of a program variable is represented by a pair (tag, value), and the tag specifies how the value must be interpreted. The setting and the checking of the tag are run time operations. The set of integers (respectively, constants) has a specific tag INT (respectively, CONST). As lists are often used, the functor ./2 has its own tag LIST, whereas all other functors are characterized by the same tag STRUCT and additional information about the actual functor. The special tag LIST avoids explicit manipulation of the functor ./2, which has to be done for all other functors.
- A general unification algorithm is used to deal with the unification of arbitrary PROLOG terms.

In a *WAM-based* compiler [28] each PROLOG clause is translated into a sequence of WAM instructions. The aim is to do as much of the work at compile time as possible in order to increase the efficiency. Here, the optimizations are based upon *local* compile time information, which can be derived directly from the PROLOG source code:

The general unification algorithm can be *specialized* for the unification between a call and the head of a particular clause. An argument in the head is often specific enough to determine at compile time which part of the general unification algorithm will be used. Special instructions are generated for such arguments. Consider the first clause of *insert*/3, "*insert*(E, *nil*, t(*nil*, E, *nil*))". As the second argument in the head is the constant *nil*, the general unification can be specialized to the unification between a constant and a PROLOG term. Similarly, the unification of the third argument is simplified to the unification between a compound term and a PROLOG term.

In order to optimize *clause selection* for a predicate call, we combine the information about the arguments in the heads of the clauses with a run time test of the tags of the arguments in the call. Again special instructions exist for this kind of optimization. In the example of *insert*/3 the second argument is suitable for indexing: if the second argument of the call has tag CONST, only the first clause can succeed (and no choice point has to be created), and if it has tag STRUCT, only the second or the third clause can succeed. A second argument with tag LIST directly implies failure. For the example of *append*/3, this optimization increases the efficiency even more. If *append*/3 is called with a given list as first argument, no choice point has to be created during its execution, as by the tag CONST (respectively, LIST) only the first (respectively, second) clause can succeed.

3.2. *Optimizations Based upon Global Compile Time Information*

We point out how further optimizations are possible using additional information about the arguments of the calls of a predicate at run time: execution steps can be detected at compile time to be redundant and can be eliminated by removing and specializing WAM instructions. But first we mention the different approaches towards obtaining this information. *Declaration-based* (*prescriptive*) systems require the user to provide declarations which characterize the predicates (e.g. mode declarations) and with which the program can be checked to be consistent. Providing the declarations requires a considerable discipline of the programmer, but they are useful for program development tools and for code generation. In *inference-based* (*descriptive*) systems all the information is inferred from the program text itself and describes the properties of the program at several points in a successful execution. Obviously, code generation can be based on the derived information, whereas programming errors are indicated by the derivation of unexpected results. Our approach is essentially inference-based, although we require declarations for the intended class of queries.

3.2.1. Modes. The importance of modes has been recognized since the use of mode declarations in the DEC-10 compiler [27], which provide information about

the instantiation state of the arguments of the calls at run time. Several mode systems have been proposed with different degrees of precision, but in the most common ones distinction is made between the mode **g** (**ground**), which specifies that the corresponding argument in any call to the predicate will always be completely instantiated; the mode **f** (**free**), which specifies the argument to be uninstantiated; and the mode **a** (**any**), indicating that there is no restriction on the argument. Note that this mode system is not *closed under substitution*: it does not have the property that if a term t has mode m, then for each concrete substitution θ it holds that $t\theta$ also has mode m. Hence, inference systems must take into account multiple occurrences of free variables in order to derive modes with a reasonable precision. The mode system with modes **a** and **g** is closed under substitution, but they are not so effective for code generation, as is clarified in the following discussion.

What optimizations can be done based on mode information? Consider the following mode information for our examples: *append*(**g**, **g**, **f**) and *insert*(**g**, **g**, **f**).

(1) Clause indexing can be based on any subset of the arguments that is either specified by the user or determined by the compiler. An argument with mode **f** should not be used for clause selection, as all the run time tests are pure overhead. An argument with mode **g** implies that the code dealing with this argument when it is free can safely be omitted. This yields simplification and compaction of the code. If an argument involved in the indexing has mode **g**, the tag test can be done once for all the clauses instead of repeating it in each clause. These optimizations are applicable in the case of *append*/3 with indexing on the first argument and of *insert*/3 with indexing on the second.

(2) The unification of a compound term in the head of a clause requires a run time test in order to decide whether its unification implies the construction of a new term (in the case of the unification with a free variable) or the unification with an existing term. The run time test can be omitted in the following two cases. If the corresponding argument in the call has mode **f**, construction is implied. This is, for instance, the case for the third argument of the second clause of *append*/3. If the corresponding argument in the call has mode **g**, unification with an existing term is implied, as for the first argument in the second clause of *append*/3.

3.2.2. Information about Possible Values of Arguments in the Calls. Mode information can be made more expressive so that it provides more precise information about possible values of a program variable, e.g. about the *principal functors* of the possible values. Consider the *append*/3 example with *nil* and ./2 as principal functors for its argument (mode **g** is still implied). The clause selection based on the first argument and the unification of the first argument with *nil*/0 or ./2 can safely be reduced to one tag test at run time. If the tag is CONST, then we know that the first argument has the value *nil*, that only the first clause has to be selected, and that the unification with *nil* succeeds. Otherwise the tag is known to be LIST; thus the second clause is selected and the unification of the first argument is reduced to dealing with the arguments of ./2. Besides increased performance, also a compaction of the code is implied. A similar optimization is possible for *insert*/3,

given that *nil* and $t/3$ are the principal functors of the second argument in the calls on which the indexing is done. One tag test suffices to distinguish between the constant *nil* and a compound term with principal functor $t/3$. Observe that if there is only one principal functor with tag STRUCT, arbitrary functors can be dealt with as efficiently as $./2$ terms with the special tag LIST.

Notice that, for instance for *append*/3, information about principal functors of the arguments of the call does not suffice to characterize the arguments of the recursive call in the body of the second clause. Therefore, we need more information which can be expressed by a *recursive type*, e.g. *List*, which denotes all lists of integers and which can be represented by[6] $List := nil|.(Int, List)$ with *Int* the type describing the set of all integers. Similarly we need a recursive type *Tree* to describe all binary trees of integers in the case of *insert*/3: $Tree := nil|t(Tree, Int, Tree)$.

In the latter case we can use the type of the elements to specialize the comparison built-ins $\leq/2$ and $>/2$ for integer arguments, whereas in general it can be any arithmetic expression. Hence, specializing built-ins is made possible by type information characterizing PROLOG terms such as integers and reals.

From the code generation point of view it is interesting to be able to distinguish sets of values that belong to one of the following classes:

1. primitive (simple, basic) types such as *Int* (set of integers), *Real* (set of reals);
2. sets of ground terms, e.g. the atom *nil*, the set of lists of integers (*List*), and the set of trees of integers (*Tree*);
3. the set of free variables;
4. sets of partially instantiated structures, such as the set of open ended lists of integers;
5. the set of all terms.

Observe that all sets in a class have the same mode: classes 1 and 2 imply mode **g**, class 3 mode **f**, and classes 4 and 5 mode **a**.

The rigid types introduced in the next section do not distinguish between classes 3 and 5, because the type *max* is used for approximating both sets. Thus, to each free variable in a term t corresponds a subterm of t which is approximated by *max*. The integrated types discussed in Section 8 have a greater expressive power and distinguish between classes 3 and 5.

4. RIGID TYPES

4.1. Definition of Normal Type Graphs

4.1.1. Graphs. Sets of values are represented by a special kind of directed graphs. First we introduce the terminology, which we adapt from graph theory.

A *directed graph* G is a pair (N, A), where N is a finite nonempty set, and A is a relation on N (A is any subset of $N \times N$). Each element in N is called a *node*, and each pair in A is called an *arc*. The arc (n, m) *leaves* the node n and *enters* the

[6] Although "Intlist" would be more appropriate, we prefer to use the shorter "List."

node m. We say that n is a *predecessor* of m, and m is a *successor* of n. PRED(n) denotes the set of predecessors of node n, and SUCC(n) denotes the successors of node n. The *indegree* of a node n is #PRED(n), and the *outdegree* of n is #SUCC(n), where #S denotes the cardinality of a set S. The arcs entering a node n are called the *incoming* arcs of n, and the arcs leaving a node n are called the *outgoing* arcs of n. A *path* is a finite sequence of one or more nodes such that, if the sequence contains two or more nodes and is denoted by $(n_1, \ldots, n_k)$ with $k \geq 2$, then $(n_i, n_{i+1}) \in A$ for $1 \leq i \leq k-1$. A *simple* path is a path with distinct nodes. If $k \geq 2$, then $(n_1, \ldots, n_k)$ is a path *from* n_1 *to* n_k with *length* $k-1$, and n_k is said to be *accessible* from n_1. As a special case, a single node denotes a path of length 0 from itself to itself. A *cycle* is a path $(n_1, \ldots, n_k)$ where $n_1 = n_k$. A *simple* cycle is an arc from a node to itself. A path is *acyclic* or *noncircular* if it does not contain a cycle.

A directed acyclic graph (acronym DAG) is a directed graph without any cycles. A (rooted) *tree* is a DAG satisfying the following properties:

(1) There is exactly one node, called the *root*, with indegree 0.
(2) Every node except the root has indegree 1.
(3) There is a path from the root to each node.

If (u, v) is an arc in the tree, then u is called the *parent* of v, and v is called a *son* of u. The *ancestor* and *descendant* relations are reflexive and transitive closures of the respective parent and son relations. Node n is called a *proper* ancestor (descendant) of node m iff n is an ancestor (descendant) of m and $n \neq m$. A *leaf* is a node n with outdegree 0.

4.1.2. Type Graphs. A *type graph* T is a triple (N, A_F, A_B) where $T_r = (N, A_F)$ is a rooted tree whose arcs in A_F are called *forward* arcs, and A_B is a restricted class of arcs, *backward* arcs, superimposed on T_r. T_r is called the *underlying* rooted tree of the type graph T. In a type graph T, TANC*(n) [respectively, TANC$^+$(n)] is the set of ancestors [respectively, proper ancestors] of the node n in the underlying rooted tree T_r. The backward arcs (n, m) in A_B have the property that $m \in$ TANC*(n). TDESC*(n) [respectively, TDESC$^+$(n)] is the set of descendants [respectively, proper descendants] of the node n in T_r. GDESC*(n) [respectively, GDESC$^+$(n)] is the set of descendants [respectively, proper descendants] of the node n in the type graph T. Note that GDESC*(n) = $\{n\} \cup$ GDESC$^+$(n) (likewise for TDESC and TANC). Observe that the set of forward descendants, TDESC, only takes into account the arcs A_F in T_r, whereas the set of descendants, GDESC, takes into account $A_F \cup A_B$. A forward path is a path composed of forward arcs.

Each node n of a type graph has a label, denoted by lb(n), indicating the kind of term it describes, and the nodes are divided into three classes:

Simple nodes have a label from the set $\{max, \bot, Int, Real, \ldots\}$, and their outdegree is 0. We give some of them a specific name: *max*-node, $\bot$-node.

Functor nodes are labeled with a functor f/k and have outdegree k with $k \geq 0$ (a constant has arity 0).

OR *nodes* have the label OR and have outdegree k with $k \geq 0$.

We use the convention that n/i denotes the ith son of node n, and the set of sons of a node n is then denoted as $\{n/1, \ldots, n/k\}$ with $k =$ outdegree(n).

Definition 4.1. A type graph $T(N, A_F, A_B)$ has the following characteristics:

- P_1. There is exactly one node, called the root n_0, with no incoming forward arc.
- P_2. All the nodes, except the root, have exactly one incoming forward arc.
- P_3. For each node, except for the root, there is exactly one path consisting of forward arcs and connecting the root with the node.
- P_4. For each backward arc $(n, m) \in A_B$, $m \in \text{TANC}^*(n)$.
- P_5. Simple nodes are leaves, functor nodes labeled f/k have outdegree k, and the outdegree of OR nodes is ≥ 0.

We assume that type graphs have disjoint sets of nodes (node names are unique over all the type graphs).

The type graphs satisfying Definition 4.1 are called *rigid types*.

The graphical representation of the type graphs is straightforward. The nodes of a type graph are represented by their labels; only the OR node is encircled, as we often drop its label. The direction of the arc is indicated by its arrow: forward arcs are drawn downwards, backward arcs upwards. The root of the type graph is the topmost node. Examples are shown in Figure 4. More convenient names are *List* for T_2, *ListOne* for T_3, *Tree* for T_4, and *TreeOne* for T_5.

4.1.3. Denotation of a Type Graph. The structure of the type graph together with the labels of its nodes determines the set of represented terms. Let S_V be a set of variables, and S_{max} the set of all the terms constructed with the functors and the constants in the program together with the variables from S_V and the terms in the primitive types (e.g. *Int*, *Real*,...). We assume that each primitive type P represents a set S_P of ground terms with depth one and that these sets are mutually disjoint. The set of finite terms represented by a node n in the type graph T is said to be the *denotation* of the node n, $\mathbb{D}(n)$.

Definition 4.2. The denotation of a node n in a type graph, $\mathbb{D}(n)$ is defined as follows:

if $\text{lb}(n) = max$ then $\mathbb{D}(n) = S_{max}$,

else if $\text{lb}(n) = \perp$ then $\mathbb{D}(n) = \varnothing$,

else if $\text{lb}(n)$ is a primitive type P then $\mathbb{D}(n) = S_P$,

else if ($\text{lb}(n) = f/k$ and $n/1, \ldots, n/k$ are its sons)

then $\mathbb{D}(n) = \{f(t_1, \ldots, t_k) | t_i$ is finite and $t_i \in \mathbb{D}(n/i)$ for $1 \leq i \leq k\}$,

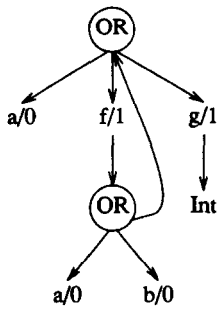
else $\mathbb{D}(n) = \bigcup_{i=1}^{k} \mathbb{D}(n/i)$, as $\text{lb}(n) = \text{OR}$ and $n/1, \ldots, n/k$ are its sons.

Note that the order of the sons of a functor node is important because they correspond to the arguments, whereas the order of the sons of an OR node is irrelevant. Observe that if the condition "t_i is finite" were dropped from the rule for $\text{lb}(n) = f/k$, then infinite terms could be included in the set due to backward arcs. $\mathbb{D}(n)$ can be $\varnothing$ or a (finite or infinite) set of finite terms. With n_0 the root of type graph T, we use $\mathbb{D}(T)$ as a synonym for $\mathbb{D}(n_0)$.

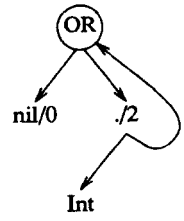
The denotations of the examples in Figure 4 are:

$\mathbb{D}(T_1) = \{a, f(a), f(b), g(1), g(2), \ldots,$
$f(f(a)), f(f(b)), f(g(1)), f(g(2)), \ldots, f(f(f(a))), \ldots\}.$

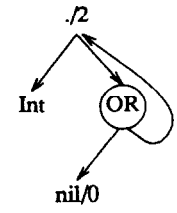
$\mathbb{D}(List) = \mathbb{D}(T_2)$ is the set of all lists of integers.



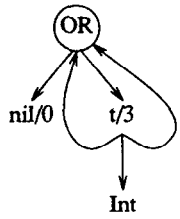
4.a Type graph T_1



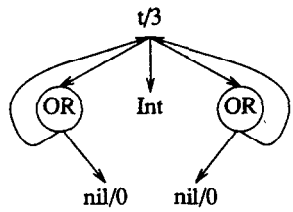
4.b Type graph T_2 ≡ List



4.c Type graph T_3 ≡ ListOne



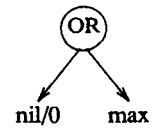
4.d Type graph T_4 ≡ Tree



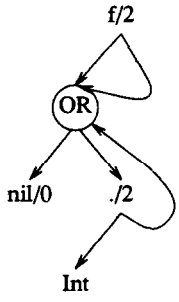
4.e Type graph T_5 ≡ TreeOne



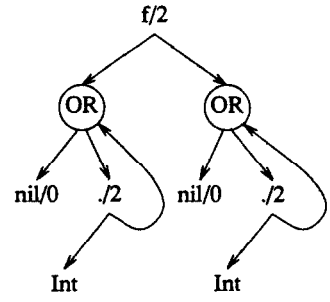
4.f Type graph T_6



4.g Type graph T_7



4.h T_8 is not a type graph



4.i Type graph T_9

FIGURE 4. Examples of type graphs.

$\mathbb{D}(ListOne) = \mathbb{D}(T_3)$ is the set of all lists of integers containing at least one element.

$\mathbb{D}(Tree) = \mathbb{D}(T_4)$ is the set of all binary trees with integer elements.

$\mathbb{D}(TreeOne) = \mathbb{D}(T_5)$ is the set of all binary trees with integer elements containing at least one element.

$\mathbb{D}(T_6) = \varnothing$.

$\mathbb{D}(T_7) = S_{max}$.

T_8 is not a type graph, as there is no underlying tree.

$\mathbb{D}(T_9) = \{f(t_1, t_2) | t_1$ and t_2 are lists of integers$\}$.

Proposition 4.1. The denotation of a rigid type is closed under substitution.

PROOF. It is sufficient to show that for each rigid type T,

$$\forall t \in \mathbb{D}(T) \quad \forall \theta: \qquad t\theta \in \mathbb{D}(T).$$

There are two cases to be considered.

(1) If t is a ground term, then $t\theta = t$ and thus $t\theta \in \mathbb{D}(T)$.
(2) If t is a nonground term, then it contains variables. For each such variable there exist a subterm t' of t and a *max*-node n in T such that the variable is a subterm of t' and t' is in the denotation of node n. This is due to the fact that according to Definition 4.2, only a *max*-node can cause the existence of nonground terms in a denotation. Note that possibly $t' = t$. Now $t'\theta \neq t'$, but $t'\theta \in \mathbb{D}(n)$, as n is a *max*-node and thus $t\theta \in \mathbb{D}(T)$. □

4.1.4. Type Graphs versus Context-Free Grammars. Context-free grammars provide an alternative way for describing sets of terms. It is straightforward to derive a context-free grammar from a type graph. For the type graphs T_1, T_2, T_3, T_4, and T_5 of Figure 4, we can derive

$$T_1 := a \mid f(a \mid b \mid T_1) \mid g(Int),$$

$$T_2 := nil \mid .(Int, T_2),$$

$$T_3 := .(Int, nil \mid T_3),$$

$$T_4 := nil \mid t(T_4, Int, T_4),$$

$$T_5 := t(nil \mid T_5, Int, nil \mid T_5).$$

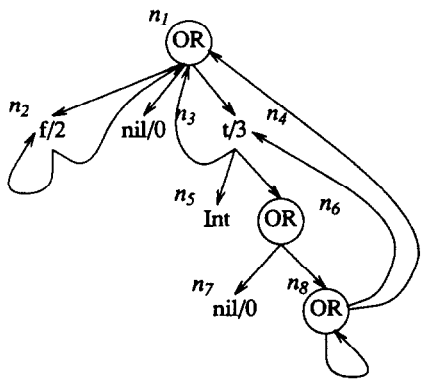
Also, deriving a type graph from a context-free grammar is rather straightforward. However, the most obvious derivation does not necessarily result in a type graph. Consider for example $T_8 := f(List, List)$ and $List := nil \mid .(Int, List)$. The most obvious graph, shown in Figure 4(h), is not a type graph, because there is no underlying tree. The proper derived type graph is shown as T_9. The underlying rooted trees were very helpful for proving the correctness of many algorithms manipulating type graphs. Moreover, they provided us with the necessary intuition for developing these algorithms. The point is that the underlying trees are explicit in the type graphs, while they are only implicit in a context-free grammar.

4.1.5. Compact Type Graphs. The type graphs are very suitable for representing a set of terms. However, several distinct type graphs can have the same denotation. This is even the case for the empty set, which can be represented not only by a $\perp$-node, but for instance also by T_6. Similarly, S_{max} can be represented by a *max*-node, but also, among others, by T_7. *Tree* is the most natural type graph describing the set of all binary trees of integers. Many others, for instance T_{10} in Figure 5, can be constructed by adding superfluous nodes and arcs without changing the denotation of the type graph, such as nodes with an empty denotation (e.g. the node n_2 in T_{10}),[7] and OR nodes, which can be safely removed without affecting the denotation (e.g. n_8 of T_{10}). The existence of these superfluous nodes and arcs makes operations needed during abstract interpretation, such as the $\leq$-operation, quite complex and inefficient, because of the increased number of nodes and arcs in the type graph, and also because detecting which set of terms is represented by a node heavily depends on the structure of the type graph. In order to reduce this variety of type graphs, additional restrictions are imposed. In a first step, *compact* type graphs are defined. The expressive power of type graphs is preserved under this restriction.

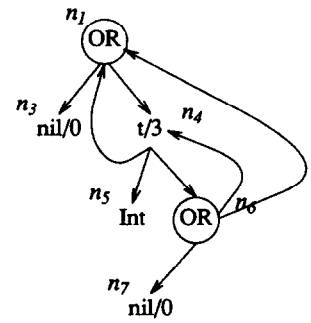
First we introduce the boolean function nonempty-$\mathbb{D}$. Its value nonempty-$\mathbb{D}(n)$ is true if $\mathbb{D}(n)$ is a nonempty set of finite terms.

Definition 4.3. nonempty-$\mathbb{D}(n)$ iff $\mathbb{D}(n) \neq \emptyset$.

An algorithm marking all nonempty nodes in a type graph can start with marking all simple nodes with a label $\neq \perp$. Subsequently, a functor node is marked when all its children are marked, and an OR node is marked when at least one of its children is marked.



5.a T_{10}
n_2 violates C_2. The arc (n_8, n_8) violates C_5. After its removal, the arc (n_6, n_8) violates C_4.



5.b compact(T_{10})
The denotation is preserved.

FIGURE 5. A nontrivial compaction.

[7] Node names are given by labels in italic added to the node labels in the type graphs.

Definition 4.4. A compact type graph $T_c(N_c, A_{Fc}, A_{Bc})$ has the following properties:

C_1. T_c is a type graph (with characteristics P_1, P_2, P_3, P_4, and P_5).
C_2. $\forall n \in N_c$: nonempty-$\mathbb{D}(n)$ or n is the root.
C_3. $\forall n \in N_c$: if lb(n) = OR then outdegree(n) > 1 and not ($\exists i$: lb(n/i) = *max*).
C_4. $\forall (n, m) \in A_{Fc}$: if lb($n$) = lb($m$) = OR then indegree($m$) > 1.
C_5. $\forall (n, m) \in A_{Bc}$: $n \neq m$, and the forward path $(m, \ldots, n)$ contains at least one functor node.

It is rather straightforward to devise an algorithm *compact*(T) that modifies the type graph T—without changing the denotation—until conditions C_1 to C_5 are satisfied (see [11]). Also, it is easy to prove that each cycle in a compact type graph T_c contains at least one OR node and at least one functor node.

The type graphs T_1, T_2, T_3, T_4, and T_5 of Figure 4 are compact. The algorithm compact(T_6) results in a $\perp$-node, as $\mathbb{D}(T_6)$ does not contain a finite term. compact(T_7) yields a type graph consisting of a single *max*-node. A less trivial case is shown in Figure 5.

Notice that compact type graphs are not the most economical representation. Nodes in different branches can have the same denotation. In particular, different sons of an OR node may have overlapping, even identical denotations, e.g. the sons n_1 and n_4 of OR node n_6 in compact(T_{10}). This makes testing whether a particular term is in the denotation of a compact type graph and the comparison of the denotations of two type graphs inefficient, so we impose further restrictions.

4.1.6. Normal and Restricted Type Graphs. The restrictions introduced here limit the expressive power of the type graphs but are necessary to achieve efficient operations and to construct a finite abstract domain (see the next section).

First we introduce the functions prnd and prlb. The function prnd(n) denotes the set of *principal nodes* of a node n, and prlb(n) its set of *principal labels*.

Definition 4.5.

$$\text{prnd}(n) = \text{if lb}(n) = \text{OR then } \bigcup_{i=1}^{k} \text{prnd}(n/i) \text{ with } k = \text{outdegree}(n), \text{ else } \{n\}.$$

$$\text{prlb}(n) = \{\text{lb}(n_i) \mid n_i \in \text{prnd}(n)\}.$$

The compactness of the type graph assures us that if lb(n) $\neq$ *max* then *max* $\neq$ prlb(n). Two sets of principal labels are *overlapping* if their intersection is nonempty.

Definition 4.6. The *principal label restriction* states that each pair of sons of an OR node must have nonoverlapping sets of principal labels.

Normal type graphs are compact type graphs satisfying the principal label restriction.

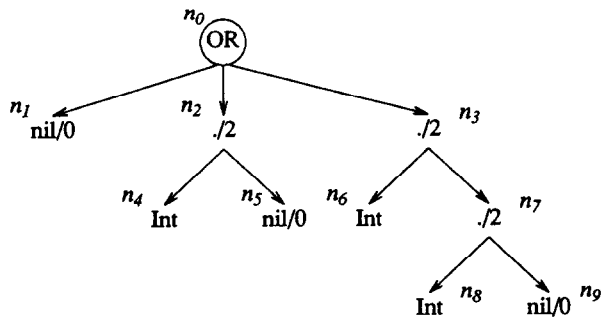
Definition 4.7. The *depth restriction* states that the number of occurrences of the same functor on each path of forward arcs in a type graph must be bounded by a constant.

Restricted type graphs are normal type graphs satisfying the depth restriction.

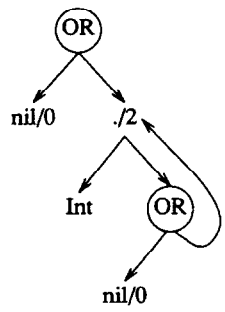
The compact type graphs T_1, T_2, T_3, T_4, and T_5 in Figure 4 do not violate these restrictions, but compact(T_{10}) does. Indeed, consider the sons of n_6: n_1 and n_4 both have $t/3$ as principal label, while n_1 and n_7 both have $nil/0$ as principal label. Type graph T_{11} of Figure 6 violates the principal label restriction at OR node n_0, as it has two sons with $./2$ as principal label, namely n_2 and n_3. If the allowed depth for $./2$ is one, then also the depth restriction is violated at node n_7.

We have developed an algorithm restrict(n). With n a node of a compact type graph T, it constructs a restricted type graph whose denotation is at least the denotation of the node n. With the depth restriction set to infinity, we obtain a variant called normalize(n) which constructs a normal type graph. We write restrict(T) and normalize(T) instead of restrict(n) and normalize(n) when n is the root of T.

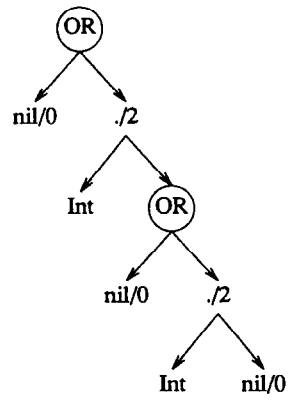
This algorithm is the first—and the most complex—of a set of similar algorithms. Algorithms in this class are given one (or two) type graphs and have to construct a new type graph satisfying certain constraints. A full description of the algorithm, with proofs of correctness and termination, can be found in [11]. The



6.a Type graph T_{11}



6.b List' = restrict(T_{11})
depth constant 1 for ./2
increased denotation



6.c normalize(T_{11})
same denotation

FIGURE 6. Normalization and restriction of a compact type graph.

appendix contains a description, together with a sketch of the correctness and termination proofs, of the similar but simpler intersection algorithm (Section 4.2.3).

The idea underlying the algorithm restrict(n) is to construct the new type graph T' step by step. The nodes of T' are divided into two sets: a set of *unexpanded leaves* requiring further processing, and a set of *safe nodes* containing all non-leaves, but also the finished leaves. The nodes of T' are labeled by a set of nodes from T. These sets are used to determine the denotation of unexpanded leaves. For example, the restricted type graph T' is initialized with a root labeled with the singleton $\{n\}$. After initialization, this root is an unexpanded leaf, and its denotation is the denotation of node n in T, i.e. the minimal denotation the restricted type graph can have.

The core of the algorithm consists of selecting an unexpanded leaf and transforming T' (addition of nodes and arcs; exceptionally, also removal to avoid violation of the depth restriction) such that:

T' remains a restricted type graph;

the denotation of nodes staying in T' never decreases.

In particular, the root is never removed, and its denotation never decreases. So, when finally the set of unexpanded leaves becomes empty, we obtain a restricted type graph whose denotation is at least the denotation of node n in T. Using set inclusion of their denotations as order relation, restricted type graphs do not have a lattice structure, so there is not always a unique best restricted type graph corresponding to a given compact type graph. This is reflected in the algorithm when it has to avoid a violation of the depth restriction. In certain cases it chooses arbitrarily between different transformations. Although each alternative leads to a solution, no one appears to be the best. We conjecture that the algorithm is optimal, i.e. that there does not exist a restricted type graph T' such that $\mathbb{D}(T') \subset \mathbb{D}(\text{restrict}(n))$ and $\mathbb{D}(T') \supseteq \mathbb{D}(n)$. Some type graphs constructed by the algorithm are shown in Figure 6 and Figure 7.

Both restrictions limit the expressiveness of the type graphs: type graphs violating these restrictions sometimes have to be replaced by a type graph denoting

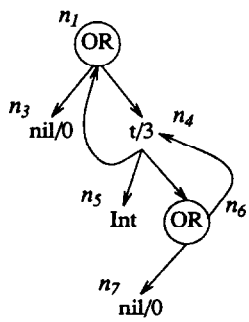


FIGURE 7. The type graph of Figure 5(b) after normalization and restriction.

Tree′ = normalize(compact(T_{10}))
Tree′ = restrict(compact(T_{10}))

a larger set of terms, e.g. $\mathbb{D}(\text{restrict}(T_{11})) \supset \mathbb{D}(T_{11})$. Note that both restrictions impose structural properties on the type graphs, which restrict their shape. However, the same set of terms can be represented by different restricted type graphs: for example, *List'* in Figure 6(b) and *List* in Figure 4(b); *Tree'* in Figure 7 and *Tree* in Figure 4(d). So no canonical form is defined for a type graph.

4.2. Operations on Normal Type Graphs

The normal type graphs will be used as a component of the abstract domains defined in the next section. The framework of Section 2 imposes an algebraic structure on the abstract domain. In this section, some of those requirements are shown to be satisfied by the normal type graphs. Proofs are given in [11]. From now on, we assume the type graphs to be normal.

4.2.1. Containment. We want to compare the denotation of two nodes in the same or two different type graphs. The algorithm $\leq(n, m, \emptyset)$ compares $\mathbb{D}(n)$ with $\mathbb{D}(m)$ and returns true iff $\mathbb{D}(n) \subseteq \mathbb{D}(m)$.

Algorithm $\leq(n, m, S^C)$.

1. if $(n, m) \in S^C$ then true
2. else if $\text{lb}(m) = max$ then true
3. else if $(\text{lb}(n) = \text{lb}(m) = f/k$ and $k > 0)$
   then $\forall i \in [1, k]$: $\leq(n/i, m/i, S^C \cup \{(n, m)\})$
4. else if $(\text{lb}(n) = \text{lb}(m) = \text{OR}$ with $k = \text{outdegree}(n))$
   then $\forall i \in [1, k]$: $\leq(n/i, m, S^C \cup \{(n, m)\})$
5. else if $(\text{lb}(m) = \text{OR}$ and $\exists m_d \in \text{prnd}(m)$: $\text{lb}(m_d) = \text{lb}(n))$
   then $\leq(n, m_d, S^C \cup \{(n, m)\})$
6. else $\text{lb}(n) = \text{lb}(m)$ % simple nodes or nodes with label $f/0$ %[8]

Proposition 4.2 (Correctness of $\leq$). $\leq(n, m, \emptyset)$ *iff* $\mathbb{D}(n) \subseteq \mathbb{D}(m)$.

Proposition 4.3. The relation $\leq$ *is reflexive and transitive.*

Proposition 4.4. There exist no infinite ascending chains under $\leq$ *in the set of restricted type graphs.*

We write $\leq(n, m, \emptyset)$ as $n \leq m$. With n_0 and m_0 the roots of the type graphs T_1 and T_2, we write $\leq(n_0, m_0, \emptyset)$ as $T_1 \leq T_2$. For example, in Figure 4 *ListOne* $\leq$ *List* and *TreeOne* $\leq$ *Tree*.

Proposition 4.5. $T_m := \max$ *is the maximal type graph.*

4.2.2. Equivalence. The equality of the denotations of the nodes n and m is computed by the algorithm $\equiv(n, m, \emptyset)$, which returns true iff $\mathbb{D}(n) = \mathbb{D}(m)$. It can be realized either as $\leq(n, m, \emptyset)$ and $\leq(m, n, \emptyset)$ or, more efficiently, by an algorithm similar in structure to the previous one. See [11] for details.

[8] A comment is added as text between two %'s.

Proposition 4.6 (Correctness of $\equiv$). $\equiv(n, m, \varnothing)$ *iff* $\mathbb{D}(n) = \mathbb{D}(m)$.

We write $\equiv(n, m, \varnothing)$ as $n \equiv m$, and, with n_0 and m_0 the roots of the type graphs T_1 and T_2, we write $\equiv(n_0, m_0, \varnothing)$ as $T_1 \equiv T_2$. Note that $\equiv$ defines an equivalence relation over type graphs, while $\leq$ modulo the equivalence classes is a partial order. For example, with *List'* of Figure 6(b) and *List* of Figure 4(b), $\equiv(List, List', \varnothing)$ returns true.

4.2.3. Intersection. This operation computes a normal type graph T_{12} which is the *intersection* of the type graphs with roots n_1 and n_2, in the sense that its denotation contains only those terms which belong to $\mathbb{D}(n_1)$ and to $\mathbb{D}(n_2)$. Note that n_1 and n_2 may actually belong to the same type graph. Due to the substitution-closedness of the denotation of rigid types, the abstract interpretation of the built-in $=/2$ is based upon the intersection operation. This algorithm, $T_{12} =$ intersection(T_1, T_2), is similar to the algorithm restrict and is described in the appendix.

Proposition 4.7. $\mathbb{D}(\text{intersection}(n_1, n_2)) = \mathbb{D}(n_1) \cap \mathbb{D}(n_2)$.

4.2.4. Backward Unification. A final algorithm that will be used is backward unification. The algorithm btunif(T_i, T_r) computes a normal type graph T_n such that

$$t \in \mathbb{D}(T_n) \quad \text{iff} \quad t \in \mathbb{D}(T_r) \text{ and } \exists t_i \in \mathbb{D}(T_i)\ \exists \sigma : t_i\sigma = t,$$

i.e., only the terms of the denotation of T_r which are instances of terms in the denotation of T_i are retained. Because the denotations of rigid types are substitution-closed, this requirement is equivalent to that for the intersection algorithm, so the latter can be used. We give this algorithm a separate name because it will be distinct from intersection in the case of the integrated types which are sketched in Section 8.

4.3. Expressive Power of the Type Graphs

A rigid type is defined by a type graph describing a set of values. Due to the imposed restrictions (the principal label restriction and the depth restriction), it is not always possible to construct, for a given set of values S, a type graph that has exactly S as its denotation: a type graph has to be used whose denotation is an overestimate of S.

The principal label restriction does not allow alternatives at an OR node with the same principal label, and (roughly said) it forces those alternatives to be assembled in one son: e.g., normalize$(f(a, b) \mid f(c, d))$ yields $T := f(a \mid c, b \mid d)$. The type systems discussed in Section 1 have similar limitations. The transformations performed on the type graph by normalize are comparable with the ones performed during the derivation of a deterministic finite automaton corresponding to a nondeterministic one [1]. The loss of precision is acceptable, as we still have information about the functor $f/2$ and its arguments. We no longer have the information that if the first argument is an a, then the second is known to be a b. Such information could be used during code generation to avoid unifications at run time.

The depth restriction is imposed to obtain a finite abstract domain. The choice of the depth constant for f/k is based on the number of occurrences of f/k in the terms appearing in the source text. This choice can be done for the whole program (taking into account the minimal value that differs from zero) or for each predicate separately, which allows finer granularity. On the one hand, the expressiveness of the type graphs is limited: e.g., if the depth constant for functor./2 is one, then *ListOne* = restrict(*OddList*) with *OddList* := .(*Int*, *nil* | .(*Int*, *OddList*)), while $\mathbb{D}(ListOne) \supset \mathbb{D}(OddList)$. On the other hand, the depth restriction enforces recursive types without any interaction with the user. The recursive types are indispensable for the detection of identical call and success substitutions.

5. ABSTRACT DOMAIN BASED UPON RIGID TYPES

5.1. Definition of Abstract Substitutions

Our aim is to describe sets of possible values of program variables at each program point. The type graphs introduced in the previous section can be used for that purpose. Notice that max has to be used to approximate the singleton containing a free variable and that any other set containing at least one nonground term is approximated by a type graph containing at least one max-node. We have a very coarse approximation of sets containing nonground terms. For this reason, we called the types *rigid*.

A program point usually contains several variables of interest. A type graph can be associated with each of them. A possible concrete substitution is then obtained by drawing for each variable a term from the denotation of its type graph. However, this does not allow one to express dependencies between variables. For example, after unifying two program variables, one can associate the same type graph with them, but this does not require that for both variables the same term be drawn to obtain a concrete substitution. Also, if another operation reduces the possible values of one variable—and modifies its type graph—one should modify the type graph of the other variable in the same way. To eliminate these sources of imprecision, an abstract substitution will not only contain type graphs but also information about which terms or subterms of the variables are *identical* in the concrete substitutions. Having this information implies that there is no difference between the outcome of the analysis for programs in the usual form and programs in their homogeneous form (predicates having distinct variables as arguments and explicit unifications) as used throughout the paper.

Before formally defining abstract substitutions, we introduce some additional notational conventions. We introduce a notation to *select* nodes in a type graph.

Definition 5.1. Let n be a node in the type graph.
n/ε selects the node n itself.
$n/(f_1/k_1, i_1).\cdots.(f_{p-1}/k_{p-1}, i_{p-1}).(f_p/k_p, i_p)$ with $1 \le i_1 \le k_1, \ldots, 1 \le i_p \le k_p$ selects the i_pth son of the principal node with label f_p/k_p of the node $n/(f_1/k_1, i_1).\cdots.(f_{p-1}/k_{p-1}, i_{p-1})$.
We call ε and $(f_1/k_1, i_1).\cdots.(f_{p-1}/k_{p-1}, i_{p-1}).(f_p/k_p, i_p)$ *selectors*.

If there are no OR nodes on the path between n and the selected node, the selector $(f_1/k_1, i_1).\cdots.(f_p/k_p, i_p)$ can be abbreviated by $i_1.\cdots.i_p$. The selector is then said to be *determinate*. If there are OR nodes on the path, the selector is said to be *nondeterminate* and the full selector is required. To select a subterm in a specific term t the abbreviated selector is sufficient; e.g., with t the term $f(g(a,b))$, $t/1.2$ selects b. We only use selectors that are well defined for each step in the selection: there always exists an appropriate principal node or term to be selected. Selectors are denoted by the letters s and p, possibly with a superscript or a subscript.

Definition 5.2. $\forall s_1, s_2$: $s_1.s_2$ is the *concatenation* of the selectors s_1 and s_2.
s_1 *extends* s_2 iff $\exists s_3$: $s_1 = s_2.s_3$.
s_1 and s_2 *overlap* iff s_1 extends s_2 or s_2 extends s_1.

The property overlap is a generalization of the property extend in the sense that it is no longer important which of the selectors extends the other.

We also introduce some notation concerning the substitutions. In the concrete substitution $\{X_i \leftarrow t, X_j \leftarrow X_j, \ldots\}$, we call t the *value* of X_i, and $X_j \leftarrow X_j$ represents the unbound variable X_j. A substitution θ *over* some domain D is a substitution with $\mathrm{dom}(\theta) = D$. T_X denotes the type associated with X in an abstract substitution. We use the symbol n_0^X for the root of a type graph T_X. $\mathbb{D}(T_X)$ is a synonym of $\mathbb{D}(n_0^X)$. In a concrete substitution terms are associated with variables belonging to its domain. X/s denotes the subterm t of the value of X such that $t \in \mathbb{D}(n_0^X/s)$. Note that the latter occurrence of s may be nondeterminate. If confusion is possible between types of X at different program points, we use T_X^β and $n_0^{X\beta}$ with β the abstract substitution at the program point. Similarly X^θ/s explicitly refers to the subterm of the value of X in the concrete substitution θ.

Definition 5.3. An abstract substitution β over a domain $D = \{X_1, X_2, \ldots, X_n\}$ is either $\perp$ or a pair (type, sval).

The TYPE *component*, type, associates a normal rigid type graph with each variable in the domain D and is selected by

$$\mathrm{TYPE}(\beta) = \{X_i \leftarrow T_{X_i} | X_i \in D \text{ and } T_{X_i} \text{ is a nonempty normal rigid type graph}\}.$$

The SVAL *component*, sval, is a set of SVAL *constraints* and is selected by SVAL(β). SVAL is an acronym of Same VALue. A SVAL constraint has the form $\{X_i/s_{X_i}, X_j/s_{X_j}\}$ with s_{X_i} and s_{X_j} determinate selectors in respectively $T_{X_i}^\beta$ and $T_{X_j}^\beta$, and it expresses the fact that $X_i^\theta/s_{X_i} = X_j^\theta/s_{X_j}$ *must* hold in any concrete substitution θ represented by β. $\{X/s_i, X/s_j\}$ is possible, but then $s_i \neq s_j$.

By convention an abstract substitution β is denoted by $\perp$ if at least one of the types T_X^β is a $\perp$-node.

Definition 5.4. A_D is the set of all abstract substitutions over the domain D.

5.2. Denotation of an Abstract Substitution

The denotation of the abstract substitution β is defined by the concretization function γ which maps β into a set Θ of concrete substitutions in which each

variable is bound to a value belonging to the denotation of its associated type graph and in which the dependencies between the values specified by the SVAL component are satisfied.

Definition 5.5. $\forall \beta \in A_D$: $\gamma(\beta) =$

if $\beta = \perp$ then $\varnothing$
else $\{\{X_1 \leftarrow t_{X_1}, \ldots, X_n \leftarrow t_{X_n}\} |$
$\{X_1 \leftarrow t_{X_1}, \ldots, X_n \leftarrow t_{X_n}\}$ is an idempotent substitution
and $\forall i$: $t_{X_i} \in \mathbb{D}(T_{X_i}^{\beta})$ and
$\forall ij$: if $\{X_i/s_{X_i}, X_j/s_{X_j}\} \in \text{SVAL}(\beta)$ then $X_i/s_{X_i} = X_j/s_{X_j}\}$.

We give some examples of abstract substitutions over the domain $\{X, Y\}$.

Example 1. $\text{TYPE}(\beta_1) = \{X \leftarrow T_X, Y \leftarrow T_Y\}$ with $T_X := f(a \mid b, a \mid c)$ and $T_Y := g(a \mid b, Int)$; $\text{SVAL}(\beta_1) = \varnothing$.

Although the SVAL component enforces no dependencies between the values of X and Y, X and Y may have identical subterms, and the following substitutions all belong to $\gamma(\beta_1)$:

$$\theta_1 = \{X \leftarrow f(a,a), Y \leftarrow g(b,3)\} \in \gamma(\beta_1),$$

$$\theta_2 = \{X \leftarrow f(a,a), Y \leftarrow g(a,3)\} \in \gamma(\beta_1),$$

$$\theta_3 = \{X \leftarrow f(a,c), Y \leftarrow g(a,3)\} \in \gamma(\beta_1).$$

Example 2. $\text{TYPE}(\beta_2) = \text{TYPE}(\beta_1)$ and $\text{SVAL}(\beta_2) = \{\{X/1, Y/1\}\}$.

The SVAL component enforces the first arguments of the values of X and Y to be identical; thus $\gamma(\beta_2) \subseteq \gamma(\beta_1)$. Now, θ_2 and $\theta_3 \in \gamma(\beta_2)$, but $\theta_1 \notin \gamma(\beta_2)$.

Example 3. $\text{TYPE}(\beta_3) = \text{TYPE}(\beta_1)$ and $\text{SVAL}(\beta_3) = \{\{X/1, Y/1\}, \{X/1, X/2\}\}$.
Now, $\theta_2 \in \gamma(\beta_3)$, but $\theta_3 \notin \gamma(\beta_3)$.

We can avoid to introduce type names for nonrecursive types that occur only once, such as T_X and T_Y in the examples above, by replacing the type names in the abstract substitution by the right hand side of their context-free grammar, e.g. $\text{TYPE}(\beta_1) = \{X \leftarrow f(a \mid b, a \mid c), Y \leftarrow g(a \mid b, Int)\}$.

5.3. *Normal Abstract Substitutions*

The TYPE component being the same, many SVAL components can yield the same $\gamma(\beta)$. One of the reasons is that equality is a transitive relation. If, for example, $\{X/s_X, Y/s_Y\}$ and $\{Y/s_Y.s, Z/s_Z\}$ belong to $\text{SVAL}(\beta)$, then $X^{\theta}/s_X.s = Z^{\theta}/s_Z$ for all $\theta \in \gamma(\beta)$. So adding $\{X/s_X.s, Z/s_Z\}$ to $\text{SVAL}(\beta)$ does not change the denotation. However, its presence is desirable (in order to have direct access to dependencies between X and Z). On the other hand, $X/s_X = Y/s_Y$ implies also $X/s_X.s = Y/s_Y.s$. Presence of the latter is undesirable, as it only enlarges the SVAL component. Finally, if $\{X/s_X, Y/s_Y\} \in \text{SVAL}(\beta)$, then the subgraphs at $n_0^{X\beta}/s_X$ and at $n_0^{Y\beta}/s_Y$ are better identical. Indeed, substitutions $\theta \in \gamma(\beta)$ have subterms $X/s_X = Y/s_Y$ drawn from the intersection of the two type graphs. The paper [11] contains

an algorithm normalize(β) which puts abstract substitutions in a normal form having all those desirable properties. The algorithm does not modify the denotation $\gamma(\beta)$.

Example. Let

$\text{TYPE}(\beta) = \{X \leftarrow f(a \mid b, a \mid c), Y \leftarrow g(a \mid b, Int)\}$,

$\text{SVAL}(\beta) = \{\{X/1, Y/1\}, \{X/1, X/2\}\}$.

$\beta_n = \text{normalize}(\beta)$ is given by

$\text{TYPE}(\beta_n) = \{X \leftarrow f(a, a), Y \leftarrow g(a, Int)\}$,

$\text{SVAL}(\beta_n) = \{\{X/1, Y/1\}, \{X/1, X/2\}, \{X/2, Y/1\}\}$.

The *normal* form of an abstract substitution is not unique: type graphs do not have a unique form; moreover, abstract substitutions that differ only in SVAL(β) can have the same $\gamma(\beta)$. Consider for example the type $T_X^{\beta_n}$ with $\#\mathbb{D}(T_X^{\beta_n}) = 1$. The SVAL component of β_n does not affect $\gamma(\beta_n)$. From now on we assume that abstract substitutions are in normal form.

5.4. *Operations on Abstract Substitutions in Normal Form*

In this section we define the algebraic structure imposed by the framework of Section 2, for the abstract domain of normal abstract substitutions. The proofs of the propositions in this section are given in [11].

Definition 5.6. A_N^D is the set of all normal abstract substitutions over domain D. (Usually D is omitted.)

5.4.1. Containment. We need an order relation $\leq$ between abstract substitutions over the same domain D satisfying the required property.

Definition 5.7. $\forall \beta, \delta \in A_N$:

$\beta \leq \delta$ iff
  $\beta = \bot$ or
  $\forall X \in D$:
    $T_X^\beta \leq T_X^\delta$ and
    $\forall \{X/s_X, Y/s_Y\} \in \text{SVAL}(\delta)\ \exists \{X/p_X, Y/p_Y\} \in \text{SVAL}(\beta)\ \exists s$:
      $s_X = p_X.s$ and $s_Y = p_Y.s$.

Proposition 5.1. $\forall \beta, \delta \in A_N$: *if* $\beta \leq \delta$ *then* $\gamma(\beta) \subseteq \gamma(\delta)$.

It is not the case that $\forall \beta, \delta \in A_N$: if $\gamma(\beta) \subseteq \gamma(\delta)$ then $\beta \leq \delta$. Consider the case of types having only one element in their denotation, e.g. $T_X^{\beta_n}$ of the previous example. The SVAL constraints do not constrain the set of concrete substitutions. Hence, for β_n' with $\text{TYPE}(\beta_n') = \text{TYPE}(\beta_n)$ and $\text{SVAL}(\beta_n') = \varnothing$ we have $\gamma(\beta_n') \subseteq \gamma(\beta_n)$, but $\beta_n' \leq \beta_n$ does not hold. One could easily avoid this by changing the definition of the normal form. The reason for not doing so is that in the abstract

domain based upon integrated types there are less trivial cases that do not satisfy this property. Failure of the test $\beta \leq \delta$ although $\gamma(\beta) \subseteq \gamma(\delta)$ has an adverse effect on the abstract interpretation algorithm: extra iterations may be required during treatment of recursive procedures, and the abstract success substitution of the recursive call may be less precise.

Proposition 5.2. $\beta_{max} = (\{X \leftarrow max | X \in D\}, \varnothing)$ *is the maximal element of* A_N.

Proposition 5.3. $\leq$ *is a reflexive and transitive relation over* A_N.

5.4.2. Equivalence.

Definition 5.8. $\forall \beta, \delta \in A_N$: $\beta \equiv \delta$ iff $\beta \leq \delta$ and $\delta \leq \beta$.

Proposition 5.4. $\forall \beta, \delta \in A_N$: *if* $\beta \equiv \delta$ *then* $\gamma(\beta) = \gamma(\delta)$.

$\leq$ is a partial order over the equivalence classes defined by $\equiv$.

5.4.3. The Upper Bound Operation

Definition 5.9. $\forall \beta_1, \beta_2 \in A_N$: $\text{upp}(\beta_1, \beta_2) =$

if $\beta_1 = \perp$ and $\beta_2 = \perp$ then $\perp$
else if $\beta_1 = \perp$ then β_2
else if $\beta_2 = \perp$ then β_1
else δ with

$\text{TYPE}(\delta) = \{X \leftarrow \text{normalize}(n_X), X \in D$ and

n_X is an OR node with two outgoing forward arcs:

one to the root of $T_X^{\beta_1}$ and one to the root of $T_X^{\beta_2}\}$,

$\text{SVAL}(\delta) = \{\{X/s_X, Y/s_Y\},$

$\beta_p, \beta_s \in \{\beta_1, \beta_2\}$ and $\beta_p \neq \beta_s$ and

$\exists \{X/s_X, Y/s_Y\} \in \text{SVAL}(\beta_s)\ \exists \{X/p_X, Y/p_Y\} \in \text{SVAL}(\beta_p)\ \exists s$:

$s_X = p_X.s$ and $s_Y = p_Y.s$ and

s_X (s_Y) is a determinate selector in T_X^{δ} (T_Y^{δ})$\}$.

$\gamma(\delta)$ includes $\gamma(\beta_1)$ and $\gamma(\beta_2)$ because the denotation of T_X^{δ} includes at least the union of the denotations of $T_X^{\beta_1}$ and $T_X^{\beta_2}$, while the SVAL component of δ expresses weaker constraints than the SVAL component of either β_1 or β_2. Indeed, $\text{SVAL}(\delta)$ contains a pair $\{X/s_X, Y/s_Y\}$ only when it occurs (explicitly or implicitly) in both β_1 and β_2 and when the selectors s_X and s_Y are still determinate in δ.

Proposition 5.5. $\forall \beta_1, \beta_2 \in A_N$: $\exists\ \text{upp}(\beta_1, \beta_2) \in A_N$: $\text{upp}(\beta_1, \beta_2) \geq \beta_1$ *and* $\text{upp}(\beta_1, \beta_2) \geq \beta_2$.

The definition of the upper bound can easily be extended for n-tuples of abstract substitutions.

5.4.4. A Finite Subdomain. We have an infinite number of normal types, so obviously we also have an infinite number of different abstract substitutions over the same domain. To obtain a finite subdomain, we impose the depth restriction on the functors. R is the operation transforming an abstract substitution to a depth-restricted abstract substitution.

Definition 5.10. $\forall \beta \in A_N$: $R(\beta) =$

if $\beta = \bot$ then $\bot$
else δ with

$\text{TYPE}(\delta) = \{X \leftarrow \text{restrict}(T_X^\beta), X \in D\}$,

$\text{SVAL}(\delta) = \{\{X/s_X, Y/s_Y\},$
$\{X/s_X, Y/s_Y\} \in \text{SVAL}(\beta)$ and
s_X and s_Y are determinate selectors
in respectively T_X^δ and T_Y^δ
and $n_0^{X\delta}/s_X \equiv n_0^{Y\delta}/s_Y\}$.

SVAL constraints are certainly removed when s_X or s_Y violates the depth bound for a functor, because the path becomes circular in δ and the selector becomes nondeterminate. Note that a SVAL component can also be removed, because the subgraph at $n_0^{X\delta}/s_X$ and at $n_0^{Y\delta}/s_Y$ do not have the same denotation. Replacing them by their intersection is not possible, because this again violates the depth restriction.

Proposition 5.6. $\forall \beta \in A_N$: $\beta \leq R(\beta)$.

Proposition 5.7. $\leq$ *over the set of depth-restricted abstract substitutions over a domain D does not have ascending chains.*

Proposition 5.8. The number of $\equiv$ *equivalence classes of depth-restricted abstract substitutions over a finite domain D for a finite set of functors is finite.*

Example.

$\text{TYPE}(\beta) = \{X \leftarrow .(T_0, .(T_1, nil)), Y \leftarrow T_0\}$,

$\text{SVAL}(\beta) = \{\{X/(./2,1), Y/\varepsilon\}\}$.

With T_0 and T_1 rigid types with nonoverlapping sets of principal functors and with 1 as depth bound of $./2$, T_X^β shown in Figure 8(a) violates the depth restriction. The computation of $R(\beta)$ first calls restrict(T_X^β), which results in T_2 shown in Figure 8(b). The SVAL constraint is dropped because $T_2/(./2,1) \equiv T_0/\varepsilon$ does not hold. Thus $\delta = R(\beta)$ is given by $\text{TYPE}(\delta) = \{X \leftarrow T_2, Y \leftarrow T_0\}$ and $\text{SVAL}(\delta) = \emptyset$.

If we retained $\{X/(./2,1), Y/\varepsilon\}$ in $\text{SVAL}(\delta)$, then $\delta_n = \text{normalize}(\delta)$ could force compatibility between the SVAL constraint and the types. However, this operation is

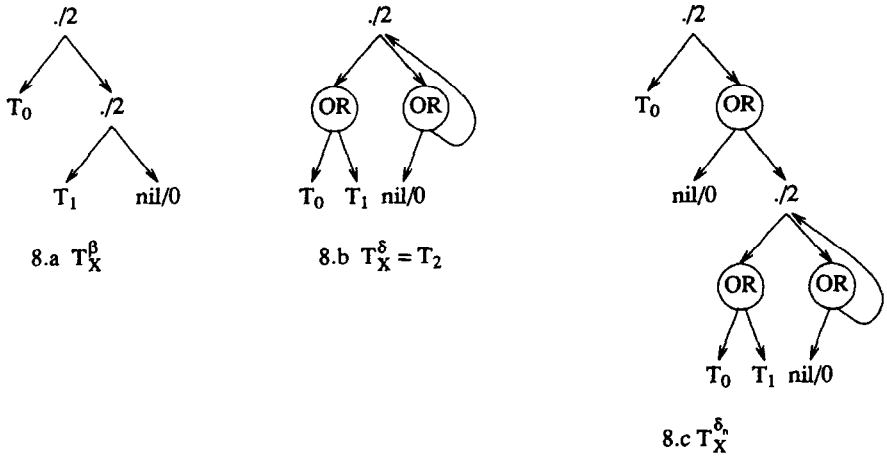


FIGURE 8. The operator R.

not valid, as $T_X^{\delta_n}$ as given in Figure 8(c) again violates the depth restriction for $./2$ and $T_2 = \text{restrict}(T_X^{\delta_n})$.

5.4.5. Backward Unification. Backward unification between β_i and β_r attempts to eliminate the concrete substitutions of $\gamma(\beta_r)$ which are not an instance of a concrete substitution in $\gamma(\beta_i)$.

Definition 5.11. $\forall \beta_i, \beta_r \in A_N$: bunif($\beta_i, \beta_r$) =

if ($\beta_i = \perp$ or $\beta_r = \perp$) then $\perp$
else normalize(β_n) with

$$\text{TYPE}(\beta_n) = \left\{X \leftarrow \text{btunif}\left(T_X^{\beta_i}, T_X^{\beta_r}\right), X \in D\right\},$$

$$\text{SVAL}(\beta_n) = \text{SVAL}(\beta_r) \cup \text{SVAL}(\beta_i).$$

Proposition 5.9. *If* $\exists \theta_r \in \gamma(\beta_r)\ \exists \theta_i \in \gamma(\beta_i)\ \exists \sigma$: $\theta_i \sigma = \theta_r$, *then* $\theta_r \in \gamma(\text{bunif}(\beta_i, \beta_r))$.

Examples. Given β_{in} and β_r,

$$\text{TYPE}(\beta_{\text{in}}) = \{X \leftarrow t(Tree, Int, max)\} \quad \text{and} \quad \text{SVAL}(\beta_{\text{in}}) = \varnothing,$$

$$\text{TYPE}(\beta_r) = \{X \leftarrow Tree\} \quad \text{and} \quad \text{SVAL}(\beta_r) = \varnothing,$$

and $\beta_n = \text{bunif}(\beta_{\text{in}}, \beta_r)$ with

$$\text{TYPE}(\beta_n) = \{X \leftarrow t(Tree, Int, Tree)\} \quad \textit{and} \quad \text{SVAL}(\beta_n) = \varnothing.$$

6. ABSTRACT INTERPRETATION OPERATIONS BASED UPON RIGID TYPES

Developing an application of the abstract interpretation framework consists not only of designing an abstract domain, but also of defining the abstract interpreta-

tion operations. They have to satisfy the correctness conditions imposed by the framework and specified in Section 2.2.2.

6.1. Procedure-Entry(P, β_{in})

Assume that $\{Y_1,\ldots,Y_n, X_1,\ldots,X_m\}$ is the domain of the call substitution β_{in} and that the call is $P(Y_1,\ldots,Y_n)$ for which a set of clauses $P(Z_1^j,\ldots,Z_n^j) \leftarrow B_1^j,\ldots,B_{j_n}^j$ exists. Assume that $\{Z_1^j,\ldots,Z_n^j,Z_{n+1}^j,\ldots,Z_q^j\}$ are the variables of the jth clause. As explained in Section 2 and Figure 3(a), Procedure-entry has to compute β_{in}^r and β_j^{in}.

Definition 6.1.

$$\text{TYPE}(\beta_{in}^r) = \{W \leftarrow T_W^{\beta_{in}} | W \in \{Y_1,\ldots,Y_n\}\},$$

$$\text{SVAL}(\beta_{in}^r) = \{\{W_1/s_1, W_2/s_2\} \in \text{SVAL}(\beta_{in}) | W_1, W_2 \in \{Y_1,\ldots,Y_n\}\},$$

$$\text{TYPE}(\beta_j^{in}) = \{Z_1^j \leftarrow T_{Y_1}^{\beta_{in}^r},\ldots,Z_n^j \leftarrow T_{Y_n}^{\beta_{in}^r}, Z_{n+1}^j \leftarrow max,\ldots,Z_q^j \leftarrow max\},$$

$$\text{SVAL}(\beta_j^{in}) = \{\{Z_k^j/s_k, Z_l^j/s_l\} | \{Y_k/s_k, Y_l/s_l\} \in \text{SVAL}(\beta_{in}^r)\}.$$

Note that we do not yield explicitly with the trivial case of β_{in} being $\perp$, as the result of any abstract interpretation operation on $\perp$ is again $\perp$.

Theorem 6.1. Procedure-entry *is correct.*

PROOF.

(1) We must show:

$$\text{if} \quad \tau \in \gamma(\beta_{in}) \quad \text{then} \quad P\tau \in \gamma(P\beta_{in}^r).$$

Assume $\tau = \{Y_1 \leftarrow t_1,\ldots,Y_n \leftarrow t_n, X_1 \leftarrow s_1,\ldots,X_m \leftarrow s_m\} \in \gamma(\beta_{in})$. Let σ be $\{Y_1 \leftarrow t_1,\ldots,Y_n \leftarrow t_n\}$. As only idempotent substitutions are considered, it is obvious that $P\sigma = P\tau$. So it suffices to show that $\sigma \in \gamma(\beta_{in}^r)$. As Y_i has the same value in σ and in τ and as $\tau \in \gamma(\beta_{in})$, it is obvious from the construction of β_{in}^r that t_i is allowed by $T_{Y_i}^{\beta_{in}}$ and that σ satisfies the SVAL constraints of β_{in}^r; thus $\sigma \in \gamma(\beta_{in}^r)$. Notice that the normal form assures that the SVAL component in β_{in}^r is as large as possible.

(2) We must prove:

$$\text{if} \quad \tau \in \gamma(\beta_{in}^r) \text{ and } \theta = \text{mgu}(P(Y_1,\ldots,Y_n)\tau, P(Z_1^j,\ldots,Z_n^j))$$

$$\text{then} \quad (P(Z_1^j,\ldots,Z_n^j) \leftarrow \cdots)\theta \in \gamma((P(Z_1^j,\ldots,Z_n^j) \leftarrow \cdots)\beta_j^{in}).$$

Assume $\tau = \{Y_1 \leftarrow t_1,\ldots,Y_n \leftarrow t_n\} \in \gamma(\beta_{in}^r)$; then $P\tau = P(t_1,\ldots,t_n)$, and $\theta = \{\cdots, Z_i^j \leftarrow t_i, \cdots\}$ is a mgu of $P(t_1,\ldots,t_n)$ and $P(Z_1^j,\ldots,Z_n^j)$. Let σ be $\{Z_1^j \leftarrow t_1,\ldots,Z_n^j \leftarrow t_n, Z_{n+1}^j \leftarrow Z_{n+1}^j,\ldots,Z_q^j \leftarrow Z_q^j\}$. Obviously $(P(Z_1^j,\ldots,Z_n^j) \leftarrow \cdots)\theta = (P(Z_1^j,\ldots,Z_n^j) \leftarrow \cdots)\sigma$. So we must show that $\sigma \in \gamma(\beta_j^{in})$. From the construction of β_j^{in} and the relationship between σ and τ, it follows that:

The values for $Z_{n+1}^j,\ldots,Z_q^j$ are in the denotation of the *max*-node.

The values for $Z_1^j, \ldots, Z_n^j$ are in the denotation of the type, because the value is identical for Y_i in τ and for Z_i^j in σ, and Y_i has the same type in β_{in}^r as Z_i^j in β_j^{in}.
τ satisfies the SVAL constraint of β_{in}^r; as the SVAL constraint of β_j^{in} is a renaming of the SVAL constraint of β_{in}^r, σ satisfies it.
Thus $\sigma \in \gamma(\beta_j^{\text{in}})$. □

6.2. Procedure-exit($P, \beta_{\text{in}}, \{\ldots, \beta_j^{\text{out}}, \ldots\}$)

The same assumptions as for Procedure-entry are made. Let β_j^{out} be the abstract success substitution of $B_{j_n}^j$. Procedure-exit has to compute β_{out}^r and β_{out} [see Figure 3(b)].

β_{out}^r over the domain $\{Y_1, \ldots, Y_n\}$ is defined as follows:

Definition 6.2.

$$\text{TYPE}(\beta'^j) = \left\{Y_1 \leftarrow T_{Z_1^j}^{\beta_j^{\text{out}}}, \ldots, Y_n \leftarrow T_{Z_n^j}^{\beta_j^{\text{out}}}\right\},$$

$$\text{SVAL}(\beta'^j) = \left\{\{Y_k/s_k, Y_l/s_l\}, \{Z_k^j/s_k, Z_l^j/s_l\} \in \text{SVAL}(\beta_j^{\text{out}}) \text{ and } k, l \in [1, n]\right\},$$

$$\beta' = \text{upp}(\beta'^1, \ldots, \beta'^p) \qquad \text{with } p \text{ the number of defining clauses,}$$

$$\beta_{\text{out}}^r = \text{bunif}(\beta_{\text{in}}^r, \beta').$$

Computing β_{out} and β_{out}^r and β_{in} (the *extension operation*) has to take into account the dependencies between variables X_q and Y_r. The possible values for such a variable X_q which did not participate directly in the call can become more restricted due to an existing dependency with a Y_r.

Definition 6.3.

$$\text{TYPE}(\beta_{\text{out}}) = \left\{\ldots, Y_j \leftarrow T_{Y_j}^{\beta_{\text{out}}^r}, \ldots, X_i \leftarrow \text{normalize}(T'_{X_i}), \ldots\right\}$$

with $T'_{X_i} =$

for each $\{X_i/s_i, Y_j/s_j\} \in \text{SVAL}(\beta_{\text{in}})$:
(not $(\exists\{X_i/s_i', Y_k/s_k\} \in \text{SVAL}(\beta_{\text{in}})$: s_i extends s_i' and $s_i' \neq s_i$)
and not $(n_0^{X_i \beta_{\text{in}}}/s_i \equiv n_0^{Y_j \beta_{\text{out}}^r}/s_j)$)
do replace$(T_{X_i}^{\beta_{\text{in}}}, s_i, \text{btunif}(n_0^{X_i \beta_{\text{in}}}/s_i, n_0^{Y_j \beta_{\text{out}}^r}/s_j))$
else $T_{X_i}^{\beta_{\text{in}}}$;

$$\text{SVAL}(\beta_{\text{out}}) = \text{SVAL}(\beta_{\text{in}}) \cup \text{SVAL}(\beta_{\text{out}}^r),$$

$$\beta_{\text{out}} = \text{normalize}(\beta_{\text{out}}).$$

The extension step looks for maximal subgraphs in $T_{X_i}^{\beta_{\text{in}}}$ for which there is an SVAL constraint with an Y_j. An update is required when the denotations of the corresponding subgraphs ($n_0^{X_i \beta_{\text{in}}}/s_i$ and $n_0^{Y_j \beta_{\text{out}}^r}/s_j$) are different. As one can only have terms which are instances of the initial terms, one can use btunif to further

restrict the denotation of the new subgraph. The details of replace, which does the actual update of the type graph of X_i, are in [11].

The careful reader might wonder why we are not also replacing the subgraph in Y_j by the result of btunif. Actually, this happens in the subsequent normalize(β_{out}) step. In fact, because btunif is equivalent with intersection for rigid types, normalize(β_{out}) could handle all updates. However, this is not the case for the integrated types of Section 8 and we have preferred to make the manipulations explicit here also.

The values of X_i may become instantiated by the call due to sharing of a free variable with a Y_j. In this case, we know that the sharing occurs in a subterm t_s such that t_s and $t_s\theta$ belong to the denotation of a max-node. So no special actions need be performed during the extension to deal with this sharing. (This is the advantage of substitution-closed types.)

The SVAL constraints in β_{out} have determinate selectors. This is trivial for those in SVAL(β^r_{out}). Those from SVAL(β_{in}) are determinate due to a property of btunif (see [11]).

Theorem 6.2. Procedure-exit *is correct.*

PROOF. Only a sketch of the proof is given; the full proof is in [11].

(1) To be proven:

$$\text{if} \quad \exists\tau_j \in \gamma\left(\beta_j^{out}\right)\ \exists\tau_{in} \in \gamma(\beta_{in})\ \exists\theta \text{ over var}(P\tau_{in}): \quad P\tau_{in}\theta = H_j\tau_j$$

$$\text{then} \quad \exists\tau \in \gamma(\beta^r_{out}): \quad P\tau = P\tau_{in}\theta.$$

As in Theorem 6.1, one assumes τ_j, τ_{in}, and θ such that $P\tau_{in}\theta = H_j\tau_j$; next one constructs a τ such that $P\tau = P\tau_{in}\theta$; finally one shows that this τ belongs to the denotation of β^r_{out}.

(2) To be proven:

$$\text{if} \quad \exists\tau_{in} \in \gamma(\beta_{in})\ \exists\theta \text{ over var}(P\tau_{in}): \quad P\tau_{in}\theta \in \gamma(P\beta^r_{out})$$

$$\text{then} \quad (H \leftarrow B_1,\ldots,P,\ldots,B_n)\tau_{in}\theta \in \gamma((H \leftarrow B_1,\ldots,P,\ldots,B_n)\beta_{out}).$$

One assumes a τ_{in} and a θ such that $P\tau_{in}\theta$ is in the denotation of $P\beta^r_{out}$; next one constructs a σ such that $(H \leftarrow B_1,\ldots)\sigma = (H \leftarrow B_1,\ldots)\tau_{in}\theta$; finally one shows that this σ belongs to the denotation of β_{out}. □

6.3. Abstract-interpretation-built-in(P, β_{in})

This operation is divided into three parts. The first part computes $\beta^r_{in} \in A_D$ and corresponds to the first part of Procedure-entry. The last part computes β_{out} from β_{in} and β^r_{out} and corresponds to the last part (extension) of Procedure-exit. So we only have to consider the middle step, computing β^r_{out} from β^r_{in}.

6.3.1. Abstract Interpretation of $X = Y$. The intended denotation of the abstract success substitution is the set

$$\{t_X\sigma, \{X \leftarrow t_X, Y \leftarrow t_Y\} \in \gamma(\beta^r_{in}) \text{ and } \sigma \text{ is a mgu of } t_X \text{ and } t_Y\}.$$

This set is contained in the set

$$\{t_X\sigma, t_X \in \mathbb{D}(T_X) \text{ and } t_Y \in \mathbb{D}(T_Y) \text{ and } \sigma \text{ is a mgu of } t_X \text{ and } t_Y\}.$$

Proposition 6.1. $\{t_X\sigma \mid t_X \in \mathbb{D}(T_X)$ *and* $t_Y \in \mathbb{D}(T_Y)$ *and* σ *is a mgu of* t_X *and* $t_Y\} = \{t \mid t \in \mathbb{D}(T_X)$ *and* $t \in \mathbb{D}(T_Y)\}$.

PROOF. This is obvious from the fact that the denotation of a rigid type is closed under substitution. □

Note that the latter set in Proposition 6.1 is just the denotation of intersection(T_X, T_Y).

Definition 6.4.

$$\text{TYPE}(\beta^r_{\text{out}}) = \{X \leftarrow T_u, Y \leftarrow T_u\} \quad \text{with} \quad T_u = \text{intersection}\left(T_X^{\beta^r_{\text{in}}}, T_Y^{\beta^r_{\text{in}}}\right),$$
$$\text{SVAL}(\beta^r_{\text{out}}) = \text{SVAL}(\beta^r_{\text{in}}) \cup \{\{X/\varepsilon, Y/\varepsilon\}\},$$
$$\beta^r_{\text{out}} = \text{normalize}(\beta^r_{\text{out}}).$$

Theorem 6.3. Abstract interpretation of $X = Y$ *is correct.*

PROOF. The proof is omitted. □

6.3.2. Abstract Interpretation of $X = f(Y_1, \ldots, Y_k)$.

Definition 6.5.

$$\text{TYPE}(\beta^r_{\text{out}}) = \left\{X \leftarrow T_u, \ldots, Y_i \leftarrow \text{normalize}\left(n_0^{X\beta^r_{\text{out}}}/(f/k, i)\right), \ldots\right\},$$

where $T_u = \text{intersection}(T_X^{\beta^r_{\text{in}}}, T_Y)$ and T_Y is a type graph with $\text{lb}(n_0^Y) = f/k$ and k sons such that $n_0^Y/i \equiv T_{Y_i}^{\beta^r_{\text{in}}}$;

$$\text{SVAL}(\beta^r_{\text{out}}) = \text{SVAL}(\beta^r_{\text{in}}) \cup \{\ldots, \{X/(f/k, i), Y_i/\varepsilon\}, \ldots\},$$
$$\beta^r_{\text{out}} = \text{normalize}(\beta^r_{\text{out}}).$$

Observe that for each Y_i we construct a new type graph for which we can prove that its denotation is the same as that of the ith son of the root of T_u. The latter is a functor node with label f/k.

Theorem 6.4. Abstract interpretation of $X = f(Y_1, \ldots, Y_k)$ *is correct.*

PROOF. The proof is omitted. □

6.4. AND-OR *Graph for insert / 3*

In this section, we illustrate the different operations with a predicate *insert*/3 which inserts an element in a binary tree. The code with names for the abstract substitutions inserted at the different program points is as follows (each clause is used only once in the AND-OR graph, so we can unambiguously name the program points in the code):

(1) *insert*(E, OT, NT):–

β_1 $\quad OT = nil,$
β_2 $\quad NT = t(nil, E, nil).$ $\beta_3.$

(2) *insert*(E, OT, NT):–

β_4 $OT = t(L, F, R)$,
β_5 $NT = t(NL, F, R)$,
β_6 $E \leq F$,
β_7 *insert*(E, L, NL). β_8

(3) *insert*(E, OT, NT):–

β_9 $OT = t(L, F, R)$,
β_{10} $NT = t(L, F, NR)$,
β_{11} $E > F$,
β_{12} *insert*(E, R, NR). β_{13}

The AND-OR graph is sketched in Figure 9; details of the third clause, which is similar to the second one, are omitted. The considered query is *insert*(X, Y, Z) with X of type *Int*, Y of type *Tree* [Figure 4(d)], and Z of type *max* (*max* used as approximation of a free variable). The SVAL component is empty.

Procedure-entry translates the information about the variables in the call into information about the variables in the heading. So for β_1, β_4, and β_9 we obtain

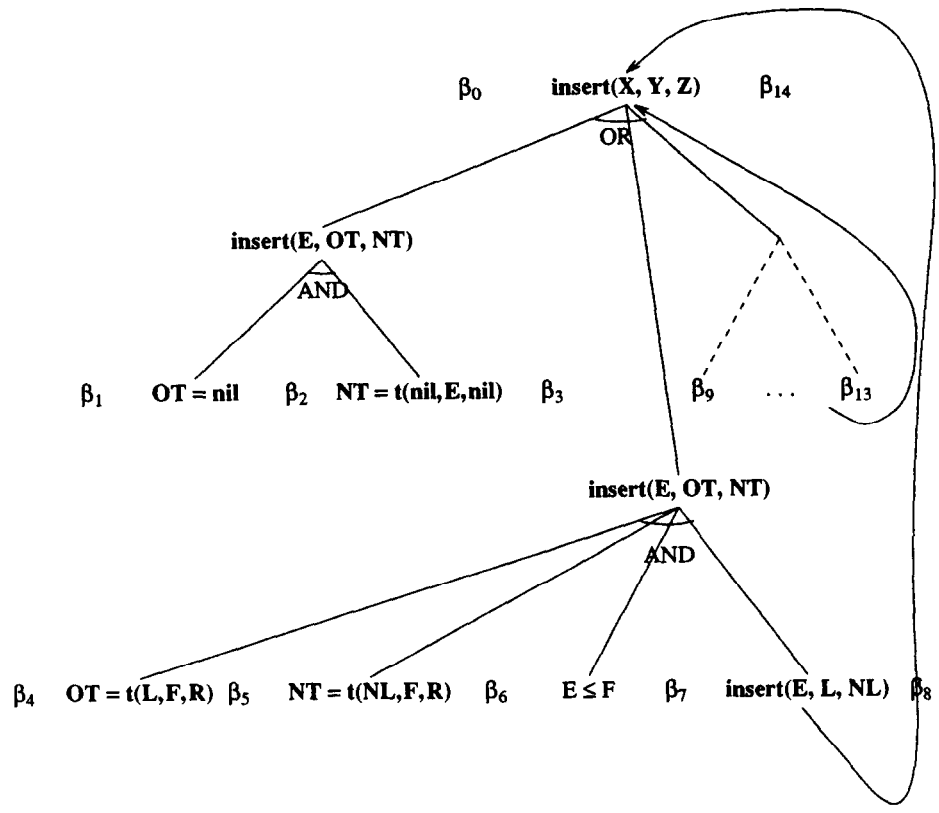


FIGURE 9. Part of the AND-OR graph for the query *insert*(X, Y, Z). (The details of the third clause are omitted.)

type *Int* for *E*, type *Tree* for *OT*, and type *max* for *NT*. The other local variables (e.g. *L*, *F*, *R*, and *NL* in the second clause) are initialized with type *max*.

Considering the first clause, the effect of the built-in $OT = nil$ is to restrict the type of *OT*: $T_{OT}^{\beta_2} := nil$. Similarly, the built-in $NT = t(nil, E, nil)$ restricts the type of *NT* but also establishes an SVAL constraint between *NT* and *E*. The resulting abstract substitution β_3 is graphically represented in Figure 10(a).

The second clause also starts with some built-ins. The effect of the call $OT = t(L, F, R)$ is to restrict the type of *L* and *R* to *Tree* and the type of *F* to *Int*, while several SVAL constraints are established. The outcome of the call $NT = t(NL, F, R)$ is that $T_{NT}^{\beta_6} = t(max, Int, Tree)$ while more SVAL constraints are created.

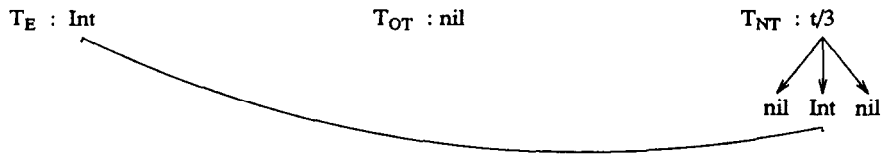


Fig. 10.a Abstract substitution β_3.

The link represents the SVAL-constraint {E/ε, NT/(t/3,2)}.

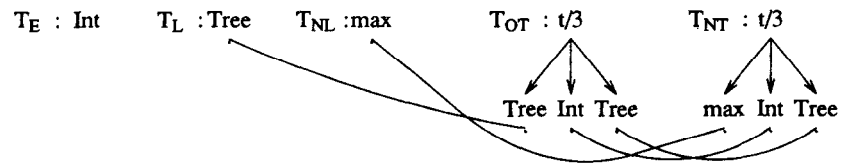


Fig. 10.b Relevant part of β_7.

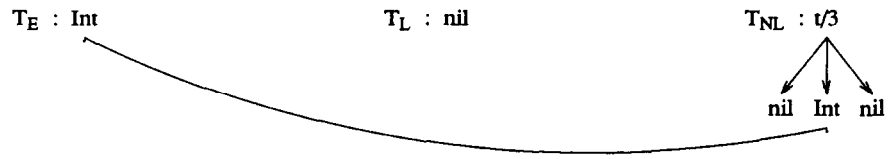


Fig. 10.c Abstract substitution β_8^r.

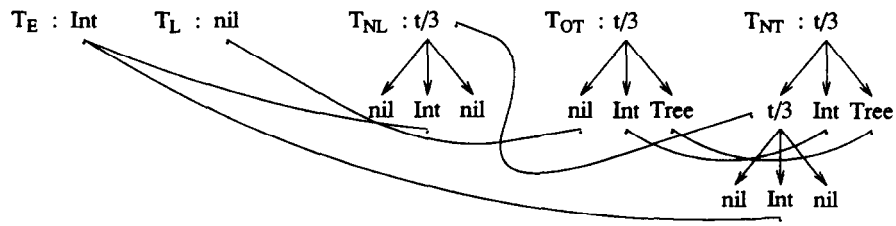


Fig. 10.d Relevant part of β_8.

FIGURE 10. Abstract substitutions of the insert/3 example.

The built-in $E \leq F$ compares two variables of type *Int* and does not modify the abstract substitution. The interesting part of β_7 is the information about the variables E, L, and NL of the recursive call and the variables (E), OT, and NT of the heading. This information is graphically represented in Figure 10(b). The information about the variables E, L, and NL in the recursive call is, up to renaming, identical to the information about the variables in the initial call. So the

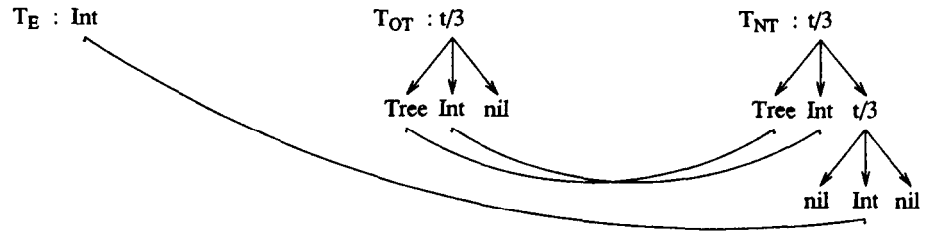


Fig. 10.e Relevant part of β_{13}.

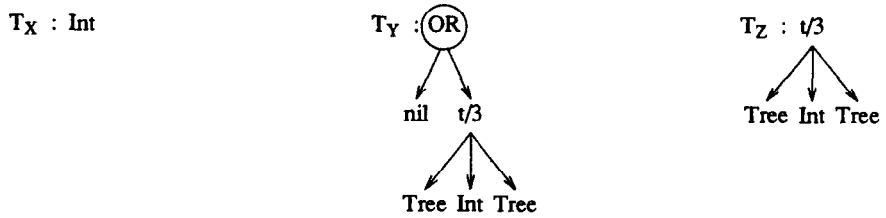


Fig. 10.f β_{14}, the success-substitution of the call.

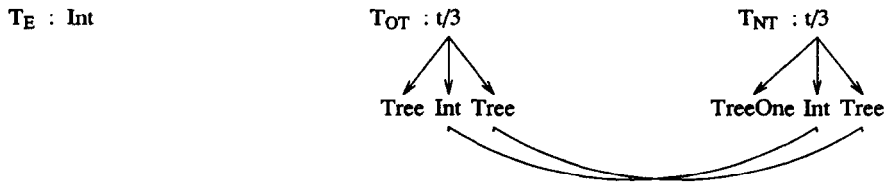


Fig. 10.g Relevant part of final β_8.

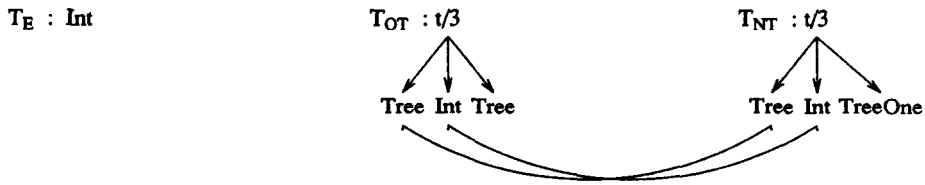


Fig. 10.h Relevant part of final β_{13}.

FIGURE 10. *(Continued)*

call is not expanded. A similar situation occurs in the branch handling the third clause.

This implies that only β_3 is available to compute a first approximation of β_{14} (β_8 and β_{13} are considered $\perp$). Procedure-exit in this case boils down to a simple renaming of β_3 and yields type *Int* for X, type *nil* for Y, and type $t(nil, Int, nil)$ for Z (and an SVAL constraint between X and Z). As this differs from $\perp$, a second iteration is required to obtain a safe approximation of β_8 and β_{13}.

β_8^r (and β_{13}^r) are the renamings of the first approximation of β_{14}. It is shown in Figure 10(c).

An extension operation is required to obtain the full β_8. The effect of extension is basically the updating of T_{NT}, which is due to the SVAL constraint between *NL* and *NT*. The SVAL constraint between *L* and *OT* causes a similar update of T_{OT}. The relevant information is shown in Figure 10(d). The processing of the third clause is very similar. The information regarding *E*, *OT*, and *NT* of β_{13} is shown in Figure 10(e).

Having new values of β_8 and β_{13}, one can again apply Procedure-exit to obtain a new approximation of β_{14} over X, Y, and Z. The inputs of the first step are shown in Figure 10(a), (d), and (e). After renaming and taking the upper bound, one obtains the situation shown in Figure 10(f). All SVAL constraints have disappeared, as there are none common to all branches. Notice that T_Y is equivalent with *Tree*, and T_Z is equivalent with *TreeOne*. In fact, the type graphs computed by the actual operations of our prototype implementation contain even more nodes. In Section 9 the issue of the size of the type graphs is discussed. Neither backward unification nor extension modifies this, so Figure 10(f) also shows β_{14}.

As the new denotation of β_{14} is larger than the previous one, another iteration is required. First of all, the restriction operator R (depth 1) is applied on T_Y and T_Z; after renaming, the obtained types of β_8^r are *Int* for *E*, *Tree* [as in Figure 4(d)] for *L*, and *TreeOne* [as in Figure 4(e)] for *NL*. The relevant part of β_8 after applying the extension step of Procedure-exit is shown in Figure 10(g) [input is β_7 as in Figure 10(b)]. β_{13} is similar, is shown in Figure 10(h). Applying Procedure-exit gives β_{14} again, as was already shown in Figure 10(f), so the computation is over.

Notice that everywhere the obtained typing is as one expects from a procedural understanding of the program. For example, the resulting type of Z is a tree with at least one element.

7. USEFULNESS OF THE DERIVED TYPES FOR CODE GENERATION

The typings obtained in the abstract call substitution for the recursive call *insert*(*E*, *L*, *NL*) were *Int* for *E*, *Tree* for *L*, and *max* for *NL*. That means that the first two arguments are ground and that all information is available for the optimizations of the code handling these arguments, as discussed in Section 3. In particular, one should use indexing on the second argument, and a single tag test is sufficient to distinguish between the two possible cases (the constant *nil*/0 and the functor *t*/3). However, optimizing the code for the third argument is not possible, as the type is *max*, and *max* corresponds to the mode **a(ny)**. In fact, the real mode is **f(ree)**, and further code improvements are possible. This could easily be detected with a mode analysis—it requires proper handling of sharing—which could be

done together with the type analysis shown. The experiment in [17] shows that if the normal execution time of *insert*/3 is used as reference and is assumed to be 100, code optimizations based upon mode information reduce the execution time to 72, and if also type information is taken into account, it is reduced to 54. Moreover, if the analysis detects that no choice point has to be created (due to the occurrence of the comparison built-ins), the respective execution times are 28 and 16. So, based on this and other examples, one can argue that rigid types are adequate for programs manipulating ground terms.

The picture is very different when considering partially instantiated data structures. Take for example insertion in an open ended tree (the following *insert*/2 detects duplicates, while the previous *insert*/3 did not; the analysis of *insert*/3 exhibits more interesting features with the version not detecting duplicates):

(1) $insert(E,T) :- T = t(L,E,R), !.$
(2) $insert(E,T) :- T = t(L,F,R),\ E < F,\ insert(E,L).$
(3) $insert(E,T) :- T = t(L,F,R),\ E > F,\ insert(E,R).$

An empty tree is represented as a free variable, so its type is *max*. Also, the type of open ended trees that can be empty is *max*. Thus, the TYPE component for a query $insert(X,Y)$ is $\{X \leftarrow Int,\ Y \leftarrow max\}$. Doing the analysis yields the same information for the recursive call. As success typing of the tree one obtains $t(max, Int, max)$. This information hardly allows any improvement of the code generated by a WAM compiler. The only useful bit of information is that the first argument of a call to *insert* is an *Int*.

Our observations for the *insert* example hold in general: rigid types do not have enough expressive power to derive adequate run time information in the case of programs using inherently partially instantiated data structures. Notice that the value of *NT* in *insert*/3 was also partially instantiated during its computation, but it became ground at the end of the computation.

However, the similarity between an open ended tree and an ordinary tree—they only differ in the value of their leaves, which are respectively free variables and *nil*/0 constants—strongly suggests that optimizations may still be applicable, if only we are able to deal with free variables as we did with constants. Therefore, we need to separate out free variables from the set of all terms. This is realized by the integrated types, which can be seen as rigid types with an additional kind of nodes, *V*-nodes, whose denotation is the set of all variables. In the next section we see how their increased expressive power affects the complexity of the abstract domain based upon them.

The abstract interpretation based upon integrated types computes the following abstract substitutions for the recursive call $insert(X,Y)$ [see Figure 11]:

abstract call substitution:
$\text{TYPE}(\beta_c) = \{X \leftarrow Int, Y \leftarrow OeTree\}, \quad \text{SVAL}(\beta_c) = \emptyset;$
abstract success substitution:
$\text{TYPE}(\beta_s) = \{X \leftarrow Int, Y \leftarrow OeTreeOne\},$
$\text{SVAL}(\beta_s) = \emptyset$
with
$OeTree := V \mid t(OeTree, Int, OeTree),$
$OeTreeOne := t(V \mid OeTreeOne, Int, V \mid OeTreeOne)$

Note that the constant *nil*/0 in the types *Tree* and *TreeOne* is simply replaced by V characterizing free variables. Consider the case that the second argument is a free variable. Based upon the type information and local analysis, we see that in the first clause a cut occurs after a unification involving a free variable. This implies that only the first clause will succeed, and the second and third clauses will never be called with a free variable as second argument. Again, one tag test allows us to distinguish between a free variable and a term with $t/3$ as principal functor. In the latter case, all clauses have to be tried.

8. ABSTRACT INTERPRETATION BASED UPON INTEGRATED TYPES

The rigid types are not sufficiently expressive to deal with partially instantiated terms in an acceptably precise manner, as illustrated by the *insert*/2 example. Moreover, the information that a variable is still free (has mode **f**), which is very important for code generation, cannot be derived from its rigid type. Therefore, we increase the expressive power of the type graphs by introducing an additional kind of nodes, the V-nodes, which explicitly represent unbound variables. Most of the terminology concerning rigid types can be used unchanged for integrated types. In order to capture the dependencies between values of variables, abstract substitutions have two additional components dealing with multiple occurrences of free variables. The abstract interpretation operations become more complex. In this section we point out the main differences with respect to the rigid types. A detailed description of the abstract interpretation based upon integrated types can be found in [11].

8.1. Integrated Types

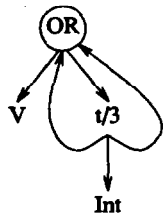
8.1.1. Definition of Integrated Types. Integrated types are type graphs as defined in Section 4.1.2 with one additional label, namely V. Nodes with label V are called *V-nodes* and belong to the class of the simple nodes. They are intended to represent unbound variables. The definition of $\mathbb{D}(n)$ can straightforwardly be extended by adding the fact that if $\mathrm{lb}(n) = V$ then $\mathbb{D}(n) = S_V$ (S_V the set of free variables occurring in S_{max}). The denotation of an integrated type is *not closed under substitution.* Consider, for instance, the integrated type $T := f(V)$ which has $f(A)$ in its denotation with $A \in S_V$. With $\sigma = \{A \leftarrow 1\}$, we have $f(A)\sigma = f(1)$ and $f(1) \neq \mathbb{D}(T)$. This implies that the definition of unification and backward unification of integrated types can no longer be based on their intersection.

All rigid types are integrated types. Type graphs describing sets of partially instantiated terms are found in Figure 11. There exist infinitely many integrated types whose denotation is S_{max}, for example:

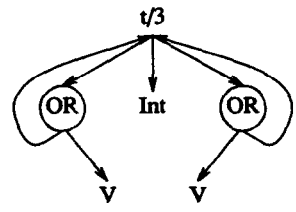
$T_m^1 := max$

$T_m^2 := V \mid Int \mid Real \mid \cdots$ (other primitive types and 0-arity functors)

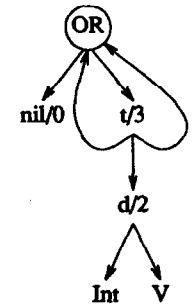
$\quad \mid f(T_m^2) \mid .(T_m^2, T_m^2) \cdots$ (a case for every functor).



11.a Type graph OeTree
ID(OeTree) is the set of all
open ended binary trees
of integers.



11.b Type graph OeTreeOne



11.c Type graph DTree
ID(DTree) is the set of all
binary trees of d/2 terms
whose 1th argument is an integer
and its 2nd a variable.

FIGURE 11. Examples of integrated types.

The term *integrated* reflects that mode information is completely subsumed by the type graphs: if the type graph consists of a single V-node, then mode **f** is implied, else if the type graph has no V-nodes or *max*-nodes, then mode **g** is implied, and otherwise mode **a** is implied. Remember that only modes **g** and **a** are derivable from rigid types.

The definitions of compact, normal, and restricted type graphs can also be used for integrated types, as well as the corresponding algorithms whose propositions remain valid.

8.1.2. Operations on Normal Integrated Types. Most operations treat V-nodes in the same way as the other simple nodes which are not max-nodes and can be used without any changes. However, an efficient version of the algorithm $n \equiv m$, which is thus not defined in terms of $\leq$, has to take into account the existence of multiple integrated types with denotation S_{max}, and a separate algorithm, similar to intersection, has to be designed for the backward unification of the integrated types T_i and T_r. The function btunif(T_i, T_r) is not symmetric in its arguments: the terms in the denotation of a subgraph of T_r can be instantiations not only of terms in the denotation of a *max*-node—as was the case for rigid types—but also of variables in the denotation of a V-node.

Example.

$\text{btunif}(t(V, Int, V), Tree) = TreeOne$ but $\text{btunif}(Tree, t(V, Int, V)) = \perp$,

$\text{btunif}(f(V \mid a), f(b \mid c)) = f(b \mid c)$,

$\text{btunif}(f(max), f(a)) = f(a)$.

8.2. Abstract Domain Based upon Integrated Types

If the set of possible values of a program variable contains nonground terms (free variables or partially instantiated terms), a better differentiation and precision is achieved using the integrated types with respect to the rigid types.

Data dependencies are again specified by the SVAL component. The identical (sub)terms can now be variables (belonging to S_V, the denotation of a V-node). These variables are said to have *multiple occurrences* or to be *shared.* In this case the sharing is forced by the SVAL component, but in general the sharing of variables between values can always occur. This means that if an arbitrary variable gets bound, all the others possibly have to be bound to the same value. This drastic and imprecise approach can only be avoided if we know which sharing is allowed or not. Therefore, a more sophisticated tracing of data dependencies is needed: we explicitly keep track of the values that are allowed to contain variables with multiple occurrences.

Definition 8.1. An abstract substitution β over a domain $D = \{X, Y, \ldots\}$ is either $\perp$ or a 4-tuple (type, sval, nuni, pshr).

The TYPE *component*, type, associates a normal integrated type graph with each variable in the domain D and is selected by TYPE(β):

$$\text{TYPE}(\beta) = \{X \leftarrow T_X \mid X \in D \text{ and } T_X \text{ is a nonempty normal integrated type graph}\}.$$

The SVAL *component*, sval, is a set of SVAL *constraints* and is selected by SVAL(β). A SVAL constraint has the form $\{X/s_X, Y/s_Y\}$ with s_X and s_Y determinate selectors in T_X^β and T_Y^β respectively, and it expresses the fact that $X^\theta/s_X = Y^\theta/s_Y$ *must* hold in any concrete substitution θ represented by β. $\{X/s_1, X/s_2\}$ is possible, but then $s_1 \neq s_2$.

The NUNI *component*, nuni, is a subset of the domain D and is selected by NUNI(β). NUNI is an acronym of NOT UNIque. The presence of X in NUNI(β) expresses the fact that the value of X is *allowed* to have multiple occurrences of a same variable in concrete substitutions represented by β (*internal sharing* of a variable).

The PSHR *component*, pshr, is a set of elements of the form $\{X, Y\}$ and is selected by PSHR(β). PSHR is an acronym of POSSIBLY SHARing. An element $\{X, Y\}$ in PSHR(β) expresses the fact that the values of X and Y are *allowed* to have occurrences of the same variable in concrete substitutions represented by β (*external sharing* of a variable).

In order to define the denotation of an abstract substitution β, the TYPE and SVAL components determine a set of concrete substitutions as in the case of the rigid types, but now concrete substitutions with multiple occurrences of a same variable are only allowed if the sharing is either forced by SVAL(β) or allowed by NUNI(β) or by PSHR(β).

Examples.

(1) Consider the abstract substitution β with domain $\{X, Y\}$:

$$\text{TYPE}(\beta) = \{X \leftarrow f(V, a \mid V), Y \leftarrow h(V, V, b)\},$$

$$\text{SVAL}(\beta) = \text{NUNI}(\beta) = \text{PSHR}(\beta) = \varnothing.$$

This specifies that the values of X and Y are not allowed to share a variable and none of the values of X or Y is allowed to have multiple occurrences of

the same variable: e.g.

$$\theta_1 = \{X \leftarrow f(A,B), Y \leftarrow h(W,T,b)\} \in \gamma(\beta).$$

The variables appearing in the concrete terms belong to $S_V = \{W, T, A, B, \ldots\}$.

(2) Extending β with SVAL$(\beta) = \{\{X/1, Y/2\}\}$ forces subterms to be identical and, as these subterms are a variable, also allows sharing: e.g.

$$\theta_2 = \{X \leftarrow f(U,T), Y \leftarrow h(W,U,b)\} \in \gamma(\beta), \qquad \text{but now} \quad \theta_1 \notin \gamma(\beta).$$

(3) Adding PSHR$(\beta) = \{\{X, Y\}\}$ allows more sharing but still no internal sharing: e.g.

$$\theta_3 = \{X \leftarrow f(U,T), Y \leftarrow h(T,U,b)\} \in \gamma(\beta) \quad \text{and still} \quad \theta_2 \in \gamma(\beta).$$

(4) Adding NUNI$(\beta) = \{X\}$ also allows internal sharing for X: e.g.

$$\theta_4 = \{X \leftarrow f(U,U), Y \leftarrow h(T,U,b)\} \in \gamma(\beta) \quad \text{and still} \quad \theta_2, \theta_3 \in \gamma(\beta).$$

Note that internal sharing is still not allowed for Y.

We prefer to keep the abstract substitutions as informative as possible. If the structure of the integrated type is such that all sharing allowed by a NUNI or a PSHR constraint is already forced by the SVAL component, the NUNI or PSHR constraint can be removed without changing $\gamma(\beta)$. The normal form computed by the algorithm normalize(β) does not contain such NUNI or PSHR constraints. However, there still can be *redundant* elements in NUNI and PSHR components of normal abstract substitutions whose removal does not change $\gamma(\beta)$. For example, if $\{X/\varepsilon, Y/\varepsilon\} \in$ SVAL(β) and $Y \in$ NUNI(β) but $X \notin$ NUNI(β), then Y in NUNI(β) is redundant. Indeed, X and Y must have the same value, so multiple occurrences of a variable in the value of Y is only possible if there are also multiple occurrences in the value of X. The latter is not allowed, so the former is not possible. Instead of removing such redundancies, we try to avoid them. We assume they are not present in the query specification. When defining operations that yield abstract substitutions, we take care to add elements to NUNI and PSHR only when they are not redundant [assuming there is no redundancy in the abstract substitution(s) which are the input(s) of the operations].

Most operations on abstract substitutions in normal form are defined as an extension of the definitions of Section 5 in order to deal with the NUNI and the PSHR component and have again the required properties. They must take into account that not only the NUNI and PSHR components express sharing, but also the SVAL component. The algebraic structure imposed by the framework is satisfied.

8.3. Abstract Interpretation Operations Based upon Integrated Types

As the denotation of an integrated type is not closed under substitution, sharing expressed by the abstract substitutions plays an important role in the definition of the abstract interpretation operations based upon integrated types. Procedure-entry and the first part of Procedure-exit are defined as straightforward extensions with respect to the rigid types, while the extension operation must deal with variables which do not appear in the call but which can become further instanti-

ated due to sharing with some of the variables actually appearing in the call. This affects not only the types of such variables but also their allowed sharing.

The extension is done in two steps. The first one merely deals with the types, and takes into account not only the SVAL constraints, but also the allowed sharing. The second step computes the sharing possibly created during the call. The call can cause new sharing between a variable appearing in the call and a nonparticipating one, or between two nonparticipating ones. The sharing is computed by an exhaustive case analysis.

The middle step of the abstract interpretation of $X = Y$ (also called the *abstract unification* of X and Y) is the unification of their type graphs T_X and T_Y. As the dependencies between the values of X and Y affect the concrete unification, they also have to be taken into account during abstract unification. The SVAL constraints specify that concrete terms must have identical components and impose an additional restriction on the result of the unification. The effect of an SVAL constraint on the abstract interpretation of $X = Y$ is clarified by the following examples.

Examples.

(1) Let

$$\text{TYPE}(\beta^r_{1\text{in}}) = \{X \leftarrow f(V,V), Y \leftarrow f(a \mid b, V)\},$$
$$\text{SVAL}(\beta^r_{1\text{in}}) = \{\{X/(f/2,1), X/(f/2,2)\}\},$$
$$\text{NUNI}(\beta^r_{1\text{in}}) = \emptyset, \qquad \text{PSHR}(\beta^r_{1\text{in}}) = \emptyset.$$

Then

$$\text{TYPE}(\beta^r_{1\text{out}}) = \{X \leftarrow f(a \mid b, a \mid b), Y \leftarrow f(a \mid b, a \mid b)\},$$

as the SVAL constraint requires the sons of the functor node $f/2$ to be identical in the case of X and due to the unification also in the case of Y. Finally,

$$\text{SVAL}(\beta^r_{1\text{out}}) = \{\{X/\varepsilon, Y/\varepsilon\}, \{X/(f/2,1), X/(f/2,2)\}, \{Y/(f/2,1), Y/(f/2,2)\}\},$$
$$\text{NUNI}(\beta^r_{1\text{out}}) = \emptyset, \qquad \text{PSHR}(\beta^r_{1\text{out}}) = \emptyset.$$

(2) Let

$$\text{TYPE}(\beta^r_{2\text{in}}) = \{X \leftarrow f(a \mid b, b), Y \leftarrow f(a \mid b, a \mid b)\},$$
$$\text{SVAL}(\beta^r_{2\text{in}}) = \{\{Y/(f/2,1), Y/(f/2,2)\}\},$$
$$\text{NUNI}(\beta^r_{2\text{in}}) = \emptyset, \qquad \text{PSHR}(\beta^r_{2\text{in}}) = \emptyset.$$

Then

$$\text{TYPE}(\beta^r_{2\text{out}}) = \{X \leftarrow f(b,b), Y \leftarrow f(b,b)\},$$

as the SVAL constraint imposes the unification of the types b and $a \mid b$, which results in the type b; and

$$\text{SVAL}(\beta^r_{2\text{out}}) = \{\{X/\varepsilon, Y/\varepsilon\}, \{X/(f/2,1), X/(f/2,2)\}, \{Y/(f/2,1), Y/(f/2,2)\}\},$$
$$\text{NUNI}(\beta^r_{2\text{out}}) = \emptyset, \qquad \text{PSHR}(\beta^r_{2\text{out}}) = \emptyset.$$

The NUNI and the PSHR constraints require an approach deviating from the treatment of variables during unification of concrete terms. Ordinary unification binds a variable to a value and replaces all occurrences of that variable by its value. A V-node in a type graph stands for one or more (if the node is reachable by a circular path) occurrences of variables in concrete terms. Also, the denotation of a *max*-node contains such variables. The sharing information tells whether variables in concrete terms are unique or can have multiple occurrences. The latter are represented by *sharable nodes*. Consider an unexpanded leaf l whose intended denotation is the set of terms resulting from the unification of terms in the denotation of nodes n_1 and n_2, with n_1 a sharable V-node. Contrary to the concrete unification, one has to consider different cases. Firstly, we have the case that n_1 does not share and thus is considered as free. The desired result is then captured by the denotation of n_2, and one can say that n_1 gets bound to n_2. Secondly, there is the case that n_1 has already a possible binding, say the node n_3. Now the desired result is captured by unifying n_3 and n_2. The trouble is that a V-node such as n_1 can have several possible bindings. Moreover, not all of them are known when dealing with l. The problem is very complicated and tricky, and a precise algorithm requires a long and complex correctness proof. So we have opted for a less precise algorithm with a simpler correctness proof (see [11] for details). We distinguish between three kinds of V-nodes:

(1) *V-nodes which are not sharable.* When unifying such a V-node with another node n, the result is the denotation of that other node. At the concrete level this corresponds to the unification of a single occurrence variable with a term.
(2) *V-nodes which are only sharable due to a* SVAL *constraint.* Abstract unification unifies a set of nodes. It happens that all V-nodes which can share due to the SVAL constraint are considered together, so this case is dealt with by taking into account the SVAL constraints.
(3) *Truly sharable V-nodes.* They are replaced by a max-node, so they become substitution-closed.

A solution, with correctness proof, treating the third class of V-nodes with more precision, can be found in [31].

The results obtained by abstract interpretation based upon integrated types for the insert/2 example were given at the end of the previous section.

9. CONCLUSIONS

Using the framework described in [3], we have developed a nontrivial application of abstract interpretation. Starting from the specification of a class of queries, the application gathers descriptions of the values program variables can have at different program points. This information can be used by a compiler to generate code which is more compact and faster in execution. Another potential application of the information is in *independent* AND *parallelism*, where goals can be executed in parallel only if they do not share common variables [8, 21].

The descriptions of possible values are in the form of abstract substitutions. The core component of an abstract substitution describes the sets of possible values of

the involved variables by means of their types. We have developed type graphs to represent types, and we use two variants. Rigid types are substitution-closed. They have the drawback that variables have to be approximated by *max*, the type with the set of all terms as denotation. Rigid types, combined with a mode analysis to find out which program variables are free, appears to be appropriate for programs dealing with ground data structures. However, they yield imprecise results for programs manipulating partially instantiated data structures such as open ended lists and open ended trees. This weakness motivated the development of integrated types. Their type graphs include a special *V*-node representing a free variable. The price to be paid is that the types are not substitution-closed and an explicit representation of sharing information is necessary. This substantially increases the complexity of algorithms and operations and of their corrections proofs, and also the processing time.

A prototype dealing with both rigid and integrated types has been developed in PROLOG. The prototype is rather slow. A typical figure for rigid types (e.g. *insert*/3) is that the global analysis takes 15 times more time than the compilation of the program with the PLM compiler written in PROLOG [24]. Using integrated types for programs manipulating open ended data structures (e.g. *insert*/2) typically requires 25 times more time than compilation.

It is worth mentioning that compile time garbage collection [20], another application of abstract interpretation, could make *insert*/3 as space- and time-efficient as *insert*/2 and would alleviate the need for using open ended data structures. Still, execution times of the prototype are rather excessive, and speedups are desirable. Several optimizations are possible:

Reducing the number of program points by bundling successive calls to the unification built-in in a single call of a more general form than discussed in the paper, e.g. $(OT, NT) = (nil, t(nil, E, nil))$ in the first clause of *insert*/3 (Section 6.4). Also, in some cases, this can eliminate some auxiliary variables.

Trimming the domains (similar to trimming environments in the WAM) of the abstract substitutions by dropping variables which occur neither in the clause head nor in the remainder of the clause body.

The current prototype represents the AND-OR graph as one data structure. It is worthwhile to investigate whether a tabular representation (as a set of records) is advantageous. It is also preferable to retain only those call substitutions for a predicate which are useful for code generation, thereby decreasing the size of the AND-OR graph and the computation time. Apparently, a simple but effective heuristic is to allow only one call substitution for each predicate on a path in the AND-OR graph.

The algorithms manipulating type graphs can be improved. In particular, algorithms such as normalize often return type graphs which are not minimal. Nonminimal type graphs have different nodes on one path with the same denotation; thus they have more nodes and arcs than necessary. This has an adverse effect on the efficiency of the operations manipulating type graphs. The reason is that a backward arc is only introduced when the simple test on

equality of two node sets succeeds. Often, this test fails because of a node whose denotation is a subset of the denotation of a node belonging to both sets. An equality test not taking into account this kind of nodes is likely to be advantageous, as it will produce smaller type graphs. Another possibility is the use of predefined types for frequently occurring type graphs such as *List* and *ListOne*. A predefined type can be represented by a simple node with a specific label, e.g. label *List*.

PROLOG systems have lots of built-ins whose abstract interpretation must be specified in the same way as for the unification $=/2$. Our application certainly benefits from a meticulous treatment of the built-ins, because their specifications can be used to steer the abstract interpretation.

Another step towards a practical system is the ability to do the analysis module by module. This requires greater effort from the programmer, as he has to declare the query patterns for all entry points of the module. Also, it has to be checked that the abstract substitution of a call to a predicate in another module is in agreement with the declaration in the other module. Disagreement can be due to imprecision in the analysis, so, instead of raising an error, including run time checks might be more appropriate. Another problem with modules is that certain predicates, e.g. append, are polymorphic: the code does not depend on the type of the elements in the lists being concatenated. With rigid types, the required extensions are more or less straightforward. One can introduce new primitive types named P_i which describe unknown collections of ground terms, i.e., the specification of calls to *append* could be *append*($List(P_1)$, $List(P_1)$, *max*), where $List(P_1)$ is defined as $nil \mid .(P_1, List(P_1))$. The main difference between types such as P_1 and types such as *Int* and *Real* is that the unification between P_1 and other types not including them as an alternative does not necessarily fail. (However, the occurrence of such unification is an indication that either the predicate is not truly polymorphic or the analysis is imprecise.) Moving to integrated types and also considering type parameters such as P_1 representing unknown collections including nonground terms has less obvious consequences regarding the sharing of variables between terms.

A simple extension of the integrated types [17] allows us to gather information about the length of the reference chains: unification of two free variables causes the creation of a reference from one variable to the other. The length of the reference chain is approximated by a lower bound and an upper bound. A similar extension of the rigid types is not appropriate, as a free variable has type max and its reference chain may have any length. Also, the combination with mode information is not useful: the knowledge that a variable is free does not give any information about the length of its reference chain.

Although a lot remains to be done, we believe that the work described is a useful contribution towards closing the performance gap between logic programming languages and conventional imperative programming languages.

APPENDIX. THE ALGORITHM INTERSECTION

The algorithm intersection(n_1, n_2) computes the type graph T_{12} whose denotation is the intersection of the denotations of the type graphs with roots n_1 and n_2. Note

that n_1 and n_2 may actually belong to the same type graph. Our strategy to deal with this kind of problem is to leave the old type graph(s) unchanged and to construct the new type graph step by step.

The initialization creates the root l_0 of T_{12} whose required denotation is defined in terms of the nodes n_1 and n_2. At this point the root l_0 is called an *unexpanded leaf*. We define the function "is" which associates at every step in the construction of T_{12} with each node in T_{12} a set of nodes from the given type graphs such that the second function on the nodes of T_{12}, $\mathbb{D}$-is, specifies for each node l of T_{12} its intended denotation.

Definition A.1. $\mathbb{D}\text{-is}(l) = \bigcap_{n \in si(l)} \mathbb{D}(n)$.

Each step extends T_{12} without decreasing the denotation of its nodes. This is done by transforming one of the unexpanded leaves l of T_{12} into a usual node (after the transformation, l is called a *safe node*), and new unexpanded leaves may be added as sons of l. The nodes of T_{12}, in each step of its construction, belong either to S^{ul}, the set of unexpanded leaves, or to S^{sn}, the set of safe nodes.

Proposition A.1. $\mathbb{D}\text{-is}(l) =$

if $(\forall n \in \text{is}(l): \text{lb}(n) = max)$ *then* S_{max}

else $\bigcap_{n \in \text{ris}(l)} \mathbb{D}(n)$ *with* $\text{ris}(l) = \{n \mid n \in \text{is}(l)$ *and* $\text{lb}(n) \neq max\}$.

Algorithm $[T_{12} = \text{intersection}(n_1, n_2)]$.

Initialization

T_{12} is initialized with a root node l_0:

$\text{is}(l_0) \leftarrow \{n_1, n_2\}$, $S^{\text{sn}} \leftarrow \varnothing$, $S^{\text{ul}} \leftarrow \{l_0\}$.

repeat

Select an l from S^{ul}, and let $\text{ris}(l)$ be the set $\{n | n \in \text{is}(l)$ and $\text{lb}(n) \neq max\}$.

1. if $\exists l_g \in S^{\text{sn}}$: $l_g \in \text{TANC}^+(l)$ and $\text{ris}(l_g) = \text{ris}(l)$ and
   $\exists l_f \in \text{TDESC}^*(l_g) \cap \text{TANC}^+(l)$: $\text{lb}(l_f) \neq \text{OR}$.
   % not all the nodes on the path from l_g to the parent of l, l_p, are OR nodes %
   then $S^{\text{ul}} \leftarrow S^{\text{ul}} \setminus \{l\}$, l is removed, and the forward arc (l_p, l) is replaced by a backward arc (l_p, l_g)
2. else if $\text{ris}(l) = \varnothing$ % $\text{is}(l)$ contains only *max* nodes %
   then $\text{lb}(l) \leftarrow max$, % each node with $\varnothing$ as ris value is a leaf in T_{12} %
   $S^{\text{ul}} \leftarrow S^{\text{ul}} \setminus \{l\}$, $S^{\text{sn}} \leftarrow S^{\text{sn}} \cup \{l\}$
3. else if $\forall n \in \text{ris}(l)$: $\text{lb}(n) = \text{OR}$
   then $\text{lb}(l) \leftarrow \text{OR}$, $S^{\text{ul}} \leftarrow S^{\text{ul}} \setminus \{l\}$, $S^{\text{sn}} \leftarrow S^{\text{sn}} \cup \{l\}$,
   select an m in $\text{ris}(l)$,
   for $i \in [1, k]$ with $k = \text{outdegree}(m)$
   do create an unexpanded leaf l/i for the ith son of l,
   $\text{is}(l/i) \leftarrow \{m/i\} \cup (\text{ris}(l) \setminus \{m\})$
   od
   $S^{\text{ul}} \leftarrow S^{\text{ul}} \cup \{l/1, \ldots, l/k\}$

4. else % $\exists n \in \text{ris}(l)$: $\text{lb}(n) = f/k$ or $\text{lb}(n) = Int, Real, \ldots$ %
   if $\forall m \in \text{ris}(l) \setminus \{n\}$: $(\exists m_d \in \text{prnd}(m)$: $\text{lb}(m_d) = \text{lb}(n))$
   then $\text{lb}(l) \leftarrow \text{lb}(n)$, $S^{\text{ul}} \leftarrow S^{\text{ul}} \setminus \{l\}$, $S^{\text{sn}} \leftarrow S^{\text{sn}} \cup \{l\}$,
      if $(\text{lb}(n) = f/k$ and $k > 0)$
      then $S_{m_d} \leftarrow \{m_d | m \in \text{ris}(l) \setminus \{n\}$ and
                $m_d \in \text{prnd}(m)$ and $\text{lb}(m_d) = \text{lb}(n)\}$
         for $i \in [1, k]$
         do create an unexpanded leaf l/i for the ith son of l
            $\text{is}(l/i) \leftarrow \{n/i\} \cup \{m/i | m \in S_{m_d}\}$
         od
         $S^{\text{ul}} \leftarrow S^{\text{ul}} \cup \{l/1, \ldots, l/k\}$
   else $\text{lb}(l) \leftarrow \perp$, $S^{\text{ul}} \leftarrow S^{\text{ul}} \setminus \{l\}$, $S^{\text{sn}} \leftarrow S^{\text{sn}} \cup \{l\}$

until $S^{\text{ul}} = \varnothing$
$T_{12} = \text{compact}(T_{12})$
% T_{12} is normal, as the principal label restriction cannot be violated. %

ListOne = intersection(*List*, *ListOne*) and *TreeOne* = intersection(*TreeOne*, *Tree*) are examples. A less trivial example is given in Figure 12.

Next, we sketch the correctness and the termination proof for the algorithm intersection. Full details are found in [11]. With respect to the correctness, we will show that $\mathbb{D}(T_{12}) = \mathbb{D}(n_1) \cap \mathbb{D}(n_2)$, but first we prove the following proposition.

Proposition A.2. $\forall m \in S^{\text{sn}}$:

if $(\text{lb}(m) = \text{OR})$ *then* $\forall t \in \mathbb{D}\text{-is}(m)\ \exists i$: $t \in \mathbb{D}\text{-is}(m/i)$ (1)

else if $(\text{lb}(m) = f/k$ *and* $k > 0)$ *then*

$\forall f(t_1, \ldots, t_k) \in \mathbb{D}\text{-is}(m)$: $(\forall i \in [1, k]$: $t_i \in \mathbb{D}\text{-is}(m/i))$ (2)

else

$\mathbb{D}(m) = \mathbb{D}\text{-is}(m)$ % *simple nodes and 0-ary functor nodes* % (3)

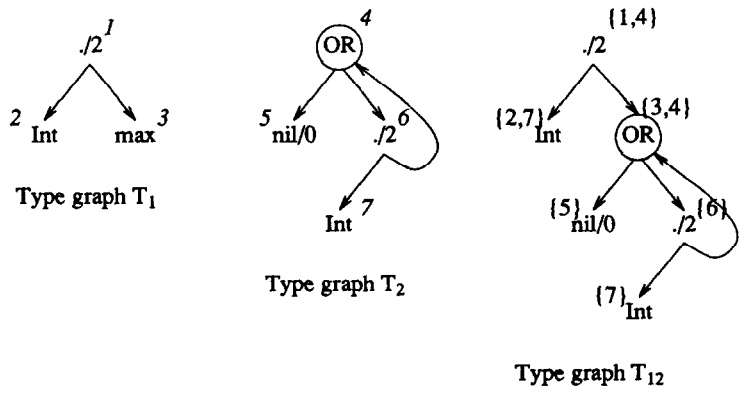


FIGURE 12. $T_{12} = \text{intersection}(T_1, T_2)$. The nodes in T_{12} are adorned with their is-values. Observe that the OR node with is-value {3,4} has ris-value {4}.

The properties (1), (2), and (3) depend only on the is-value of the node and the is-value of its children (if any). So this proposition can be proved by considering for each transformation step the nodes for which these sets are modified and by showing that their corresponding properties still hold after the transformation due to the properties of the construction and Proposition A.1. A detailed proof is found in [11].

Proposition A.3. If $T_{12} = \text{intersection}(n_1, n_2)$ *then* $(t \in \mathbb{D}(T_{12})$ *iff* $t \in \mathbb{D}(n_1)$ *and* $t \in \mathbb{D}(n_2))$.

PROOF. Initially we have that $\text{is}(l_0) = \{n_1, n_2\}$, and since no transformation step changes the value of $\text{is}(l_0)$, we have that $\mathbb{D}\text{-is}(l_0) = \mathbb{D}(n_1) \cap \mathbb{D}(n_2)$. It remains to prove that $\mathbb{D}(l_0) = \mathbb{D}\text{-is}(l_0)$. Therefore, we show that

$$\forall l \in T_{12}: (t \in \mathbb{D}(l) \text{ iff } t \in \mathbb{D}\text{-is}(l)) \tag{4}$$

1. $\forall t \in \mathbb{D}\text{-is}(l)$: $t \in \mathbb{D}(l)$.

   Base: The depth of the term t is 1. (4) immediately follows from (3) when l is a simple node or a 0-arity functor node. There remains the case that l is an OR node; then t should belong to the denotation of one of its sons. According to (1) there exists an i such that t belongs to $\mathbb{D}\text{-is}(l/i)$. We show that t belongs to $\mathbb{D}(l/i)$, the denotation of the ith son of l. Either l/i is a simple node or it is a 0-arity functor node and we are finished, or it is again an OR node. However, a cycle consisting of only OR nodes cannot occur, due to the condition imposed at the creation of a backward arc. Thus, eventually a non-OR node is reached.

   Step: $t = f(t_1, \ldots, t_k)$ *with* h *the depth of the term* t *and* $h > 1$. With l a functor node f/k, we have to show that $\forall i \in [1, k]$: $t_i \in \mathbb{D}(l/i)$. From (2) we derive $t_i \in \mathbb{D}\text{-is}(l/i)$. The depth of t_i is less than h, and from the induction hypothesis it follows that $t_i \in \mathbb{D}(l/i)$. With l an OR node, we apply the same reasoning as in the base case. Eventually, we reach a functor node and reduce the depth of the term; thus the induction hypothesis can be applied.

2. $\forall t \in \mathbb{D}(l)$: $t \in \mathbb{D}\text{-is}(l)$. Again, the proof is by induction on the depth of the terms. If term t has depth 1 and $t \in \mathbb{D}(l)$, then we can prove, for each kind of node l which possibly has t in its denotation, that $t \in \mathbb{D}\text{-is}(l)$. If the depth of t is > 1, then t belongs to the denotation of some functor node. We use the induction hypothesis to prove the property for its sons. Together with the characteristics of the construction this allows us to prove the property for the functor node itself. Details are found in [11]. □

Proposition A.4. The algorithm intersection *terminates.*

PROOF. Termination is proved through well-foundness by using a theorem from Manna and Ness [16], which has provided the basis for most techniques used for proving termination of production systems. Theorem A.1 is a reformulation of the theorem for the algorithm intersection with $S_{T'}$ the set of successive intermediate states of the new graph $\{T'_0, \ldots, T'_i, T'_{i+1}, \ldots\}$.

Definition A.2. A partially ordered set $(F, <_F)$ is said to be *well founded* if there is no infinite increasing sequence of elements $f_1 <_F f_2 <_F \cdots$ from the set F.

Theorem A.1. The algorithm intersection *terminates if there exists a well-founded set* $(F, <_F)$ *and a function* $\tau : S_{T'} \to F$ *(called the termination function) such that if* T'_{i+1} *is the intermediate state that follows immediately on* T'_i, *then* $\tau(T'_i) <_F \tau(T'_{i+1})$.

PROOF. See [16] □

In the remaining part of this appendix we prove Theorem A.1 and thus Proposition A.4 by defining a well-founded set F and a termination function τ.

Definition A.3. A path $(l_0, \ldots, l_{j-1}, l_j)$ in a graph $T \in S_{T'}$ is a *maximal path* iff either $(l_0, \ldots, l_j)$ is a forward path with outdegree$(l_j) = 0$, or (l_{j-1}, l_j) is the only backward arc.

$P(T) = \{p \mid p$ is a maximal path of $T\}$.

$\omega_T(p) = \text{ris}(l_0).\text{ris}(l_1).\cdots.\text{ris}(l_j).\text{nil}$ is the *weight* of the path $p = (l_0, l_1, \ldots, l_j)$ in T.

% If no confusion is possible, the subscript T is dropped. %

$W(T) = \{\omega(p) \mid p \in P(T)\}$ is the *weight of the graph* T.

$F = \{(W(T'_i), \#S^{\text{ul}}_{T'_i}) \mid T'_i \in S_{T'}$ and $\#S^{\text{ul}}_{T'_i}$ is the number of elements in S^{ul} of $T'_i\}$.

$\tau : S_{T'} \to F : T'_i \to (W(T'_i), \#S^{\text{ul}}_{T'_i})$.

% Now we can define the strict order relation $<_F$ over F. %

$(W(T_1), \#S^{\text{ul}}_{T_1}) <_F (W(T_2), \#S^{\text{ul}}_{T_2})$ iff

$$W(T_1) <_W W(T_2) \quad \text{or} \quad \left(W(T_1) =_W W(T_2) \text{ and } \#S^{\text{ul}}_{T_1} > \#S^{\text{ul}}_{T_2}\right).$$

$W(T_1) <_W W(T_2)$ iff $W(T_1) \leq_W W(T_2)$ and not $W(T_2) \leq_W W(T_1)$.

$W(T_1) \leq_W W(T_2)$ iff

$$\forall p \in P(T_1) \cap P(T_2): \qquad \omega_{T_1}(p) \leq_\omega \omega_{T_2}(p)$$

and

$$\forall p_2 \in P(T_2) \setminus P(T_1) \quad \exists p_1 \in P(T_1) \setminus P(T_2): \qquad \omega_{T_1}(p_1) \leq_\omega \omega_{T_2}(p_2).$$

$W(T_1) =_W W(T_2)$ iff $W(T_1) \leq_W W(T_2)$ and $W(T_2) \leq_W W(T_1)$.

$\omega_{T_1}(p_1) \leq_\omega \omega_{T_2}(p_2)$ iff

$$\omega_{T_1}(p_1) = \text{nil}$$

or

$$\omega_{T_1}(p_1) = \text{ris}.\omega'_{T_1}(p_1) \quad \text{and} \quad \omega_{T_2}(p_2) = \text{ris}.\omega'_{T_2}(p_2)$$

$$\text{and} \quad \omega'_{T_1}(p_1) \leq_\omega \omega'_{T_2}(p_2).$$

Lemma A.1. Each step of algorithm intersection *strictly increases the value of* τ.

PROOF (Sketch). Each step either replaces a path by at least one path with a strictly increased weight, or does not change the weight of the graph but decreases $\#S^{\text{ul}}$. For instance, step 1 replaces the path $(l_0, \ldots, l_g, \ldots, l)$ by a path $(l_0, \ldots, l_g, \ldots, l_g)$. As $\text{ris}(l) = \text{ris}(l_g)$, the weight of this path (and all other paths) is unchanged. Thus the weight of the graph remains unchanged. However, $\#S^{\text{ul}}$ decreases, so τ increases.

Step 3 replaces a path $p_1 = (l_0, \ldots, l_j)$ by at least one path $p_2 = (l_0, \ldots, l_j, l_j/i)$. In the new graph, $\text{ris}(l_j)$ did not change, so $\omega(p_1) \leq_\omega \omega(p_2)$ and moreover $\omega(p_1) \neq \omega(p_2)$ for all p_2; all other paths keep the same weight. Thus the weight of the graph strictly increases, and also the value of τ. □

Lemma A.2. $(F, <_F)$ *is a well-founded set.*

PROOF (Sketch). The elements in F are tuples which are determined by the graphs in $S_{T'}$. This implies that the properties of F can be formulated in terms of the successive intermediate graphs. In order to show that $(F, <_F)$ does not contain an infinite, increasing sequence, it suffices to show that there exists a fixed, finite upper bound, UB_1, for the number of nodes in any is-value and that there exists a fixed, finite upper bound, UB_2, for the length of the maximal paths. Let lbs be the set of distinct labels occurring in the given type graphs. $\text{UB}_1 = \text{GDESC}^*(n_1) \cup \text{GDESC}^*(n_2)$ and $\text{UB}_2 = \#\text{lbs} \times \#2^{\text{UB}_1}$ can be shown to be valid upper bounds. □

We are indebted to Anne Mulkers, John Gallagher, and the referees for many useful comments on a first version of the paper. During the revision, we got many helpful suggestions from Danny De Schreye, John Gallagher, and Saumya Debray, while Anne Mulkers did some meticulous proofreading.

REFERENCES

1. Aho, A., Hopcroft, J., and Ullman, J., *The Design and Analysis of Computer Algorithms*, Addison-Wesley, 1974.
2. Bansal, A., and Sterling, L., An Abstract Interpretation Scheme for Logic Programs Based on Type Expression, in: *Proceedings of the International Conference on Fifth Generation Computer Systems*, Tokyo, 1988, pp. 422–429.
3. Bruynooghe, M., A Practical Framework for the Abstract Interpretation of Logic Programs, *J. Logic Programming* 10:91–124 (1991).
4. Bruynooghe, M., and Janssens, G., An Instance of Abstract Interpretation Integrating Type and Mode Inferencing, in: *Proceedings of the 5th International Conference and Symposium on Logic Programming*, Seattle, 1988, pp. 669–683.
5. Bruynooghe, M., Janssens, G., Callebaut, A., and Demoen, B., Abstract Interpretation: Towards the Global Optimization of Prolog Programs, in: *Proceedings of the 4th Symposium on Logic Programming*, San Francisco, 1987, pp. 192–204.
6. Debray, S. K., and Warren, D. S., Automatic Mode Inference for Prolog Programs, in: *Proceedings of the 3rd Symposium on Logic Programming*, Salt Lake City, 1986, pp. 78–88.
7. Hanus, M., Polymorphic Higher-Order Programming in Prolog, in: *Proceedings of the 6th International Conference on Logic Programming*, Lisbon, 1989, pp. 382–397.
8. Jacobs, D., and Langen, A., Accurate and Efficient Approximation of Variable Aliasing in Logic Programs, in: *North American Conference on Logic Programming*, Cleveland, 1989, pp. 154–165.
9. Janssens, G., Deriving Run Time Properties of Logic Programs by Means of Abstract Interpretation, Ph.D. Thesis, Dept. of Computer Science, K. U. Leuven, Mar. 1990.
10. Janssens, G., and Bruynooghe, M., An Application of Abstract Interpretation: Integrated Type and Mode Inferencing, Report CW86, Dept. of Computer Science, K. U. Leuven, 1989.
11. Janssens, G., and Bruynooghe, M., Deriving Descriptions of Possible Values of Program Variables by Means of Abstract Interpretation: Definitions and Proofs, Report CW108, Dept. of Computer Science, K. U. Leuven, 1990.

12. Kanamori, T., and Horiuchi, K., Type Inference in Prolog and Its Application, in: *Proceedings of the 9th International Joint Conference on Artificial Intelligence*, Los Angeles, 1985, pp. 704–707.
13. Kanamori, T., and Kawamura, T., Analyzing Success Patterns of Logic Programs by Abstract Hybrid Interpretation, ICOT Technical Report TR 279, 1987.
14. Kluzniak, F., Type Synthesis for Ground Prolog, in: *Proceedings of the 4th International Conference on Logic Programming*, Melbourne, 1987, pp. 788–816.
15. Manilla, H., and Ukkonen, E., Flow Analysis of Prolog Programs, in: *Proceedings of the 4th Symposium on Logic Programming*, San Francisco, 1987, pp. 205–214.
16. Manna, Z., and Ness, S., On the Termination of Markov Algorithms, in: *Proceedings of the 3rd Hawaii International Conference on Systems Science*, Honolulu, 1970, pp. 789–792.
17. Mariën, A., Janssens, G., Mulkers, A., and Bruynooghe, M., The Impact of Abstract Interpretation on Code Generation: An Experiment in Efficiency, in: *Proceedings of the 6th International Conference on Logic Programming*, Lisbon, 1989, pp. 33–47.
18. Mellish, C. S., The Automatic Generation of Mode Declarations for Prolog Programs, DAI Research Paper 163, Dept. of Artificial Intelligence, Univ. of Edinburgh, 1981.
19. Mishra, P., Towards a Theory of Types in Prolog, in: *Proceedings of the 1st Symposium on Logic Programming*, Atlantic City, 1984, pp. 289–298.
20. Mulkers, A., Winsborough, W., and Bruynooghe, M., Analysis of Shared Data Structures for Compile-Time Garbage Collection in Logic Programs, in: *Proceedings of the 7th International Conference on Logic Programming, Jerusalem*, 1990, pp. 747–762.
21. Muthukumar, K., and Hermenegildo, M., Determination of Variable Dependence Information at Compile Time through Abstract Interpretation, in: *North American Conference on Logic Programming*, Cleveland, 1989, pp. 166–185.
22. Mycroft, A., and O'Keefe, R. A., A Polymorphic Type System for Prolog, *Artif. Intell.* 23:295–307, 1984.
23. Nilsson, U., Inferring Restricted AND-Parallelism in Logic Programs Using Abstract Interpretation, Report Li TH-IDA-R-88-02, Linköping Univ., 1988.
24. Van Roy, P., A Prolog Compiler for the PLM, Master's Report Plan II, Computer Science Div., Univ. of California, Berkeley, 1984.
25. Somogyi, Z., A System of Precise Modes for Logic Programs, in: *Proceedings of the 4th International Conference on Logic Programming*, Melbourne, 1987, pp. 769–787.
26. Sondergaard, H., An Application of Abstract Interpretation of Logic Programs: Occur Check Reduction, in: *Proceedings of the European Symposium on Programming*, Saarbrücken, 1986, pp. 327–338.
27. Warren, D. H. D., Implementing Prolog—Compiling Predicate Logic Programs, DAI Research Reports 39 40, Dept. of Artificial Intelligence, Univ. of Edinburgh, 1977.
28. Warren, D. H. D., An Abstract Prolog Instruction Set, Technical Report, Artificial Intelligence Center, SRI International, 1983.
29. Warren, R., Hermenegildo, M., and Debray, S., On the Practicality of Global Flow Analysis of Logic Programs, in: *Proceedings of the 5th International Conference and Symposium on Logic Programming*, Seattle, 1988, pp. 684–699.
30. Winsborough, W., Path-Dependent Reachability Analysis for Multiple Specialization, in: *North American Conference on Logic Programming*, Cleveland, 1989, pp. 133–153.
31. Winsborough, W., and Bruynooghe, M., Approximating Unification over Representations of Sets of Nonground Terms, in preparation.
32. Winsborough, W., and Waern, A., And-Parallelism in the Presence of Shared Free Variables, in: *Proceedings of the 5th International Conference and Symposium on Logic Programming*, Seattle, 1988, pp. 749–764.
33. Xu, J., and Warren, D. S., A Type Inference System for Prolog, in: *Proceedings of the 5th International Conference and Symposium on Logic Programming*, Seattle, 1988, pp. 604–619.
34. Yardeni, E., and Shapiro, E., A Type System for Logic Programs, in: E. Shapiro (ed.), *Concurrent Prolog: Collected Papers*, MIT Press, Cambridge, Mass., 1987, pp. 211–244.
35. Zobel, J., Derivation of Polymorphic Types for Prolog Programs, in: *Proceedings of the 4th International Conference on Logic Programming*, Melbourne, 1987, pp. 817–838.